El mejor apunte de TBD que vas a ver en tu vida

Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Escuela de Ciencias Exactas y Naturales

Juan Figueredo, Mateo Bacci, Luciano Scola, Bautista Peirone

29 de enero de 2024

Índice

1. Introducción **4**

2. Modelo Entidad Relación **5**

2.1. Entidades 5
2.2. Atributos 5
2.3. Relaciones 6
2.4. Restricciones de asignación (mapping) 6
2.5. Claves en entidades 7
2.6. Claves en relaciones 8
2.7. Diagrama entidad-relación 8
2.8. Reducción de DER a tablas (mapa canónico) 9
2.9. Generalizaciones y especializaciones 10

3. Modelo Relacional **11**

3.1. Esquemas e instancias 11
3.2. Claves 12

4. DDLs, DMLs y SQL **13**

4.1. ¿Qué son los DDL y DML? 13
4.2. ¿Y SQL? 13
4.3. Índices 17
4.4. Otras funcionalidades 17

5. Álgebra Relacional **19**

5.1. Operaciones fundamentales 19
5.2. Operaciones adicionales 21

6. Cálculo Relacional **24**

6.1. Cálculo relacional de tuplas 24
6.2. Cálculo relacional de dominios 25
6.3. Completitud relacional 26
6.4. Comparación de lenguajes algebraicos y de cálculo 26

7. Optimización **28**

7.1. Necesidad 28

7.2. Proceso de optimización 29

8. Integridad **33**

8.1. Restricciones de dominio 33

8.2. Restricciones de integridad de las entidades 33

8.3. Restricciones de integridad referencial 34

8.4. Como manejar las claves ajenas 35

9. Seguridad **37**

9.1. Vistas 37

9.2. Subsistema de autorización 38

10. Catálogo **40**

10.1. Obtención de Información 40

10.2. Características 41

10.3. Estructura y Contenido 41

10.4. Ejemplos 42

10.5. Consultas 43

10.6. Actualización 44

11. Dependencias Funcionales **46**

11.1. ¿Qué es una Dependencia Funcional? 46

11.2. Cierre y Axiomas de Armstrong 47

11.3. Proyección y Preservación de Dependencias 48

11.4. Recubrimiento canónico 48

12. Normalización **50**

12.1. Formas Normales y Dependencias Funcionales 50

12.2. Mal Diseño de Bases de Datos 50

12.3. Primera Forma Normal 52

12.4. Segunda Forma Normal 53

12.5. Tercera Forma Normal 54

12.6. Pérdida de Información 55
12.7. Pérdida de Dependencias Funcionales 56
12.8. Forma Normal de Boyce-Codd 56

13.Recuperación **58**
13.1. Manejador de transacciones 58
13.2. Punto de sincronización 59
13.3. Procotolo de bitácora de escritura adelantada y Punto de Revisión 59
13.4. Fallas 60

14.Concurrencia **62**
14.1. Problemas de concurrencia 62
14.2. Bloqueos 63
14.3. Bloqueo Mutuo 64

15.Bases de Datos Distribuidas **65**
15.1. Ventajas y desventajas 65
15.2. Reglas 66

16.Bibliografía **69**

1. Introducción

El único propósito de la existencia de este documento es conseguir los conocimientos **justos y necesarios** para aprobar la materia de "*Teoría de Bases de datos*".
El resumen está dividido en **unidades independientes** y fue escrito por muchas manos en muchos momentos distintos, por lo que no se asegura un hilo conductor entre tema y tema.
Esperamos que les sirva,
Toto, Bati, Lucho y Juan.

PD: Soundtrack para leer el apunte.

2. Modelo Entidad Relación

Un **modelo de datos** es una colección de herramientas conceptuales que usamos para describir datos, su semántica asociada, las relaciones que existen entre ellos, y restricciones de consistencia.
En esta materia utilizamos dos modelos distintos: el modelo entidad-relación y el modelo relacional (se explica en la sección 3).
El **modelo entidad-relación** (MER a partir de ahora) es una técnica de diseño de BD basado en objetos que se usan para describir datos a **nivel conceptual**. Tiene tres conceptos principales: entidades, atributos y relaciones.

2.1. Entidades

Una **entidad** es un objeto que existe y es distinguible de otros objetos. Puede ser:

- **Concreta**: persona, empleado, casa, auto, etc.
- **Abstracta**: cuenta bancaria, empresa, curso, etc.

Lo que representa a una entidad y la diferencia del resto son sus **atributos**.

2.2. Atributos

Un **atributo** es una propiedad específica que describe entidades. Por ejemplo, los atributos de una persona pueden ser su edad, número de documento, nombre, sexo, etc. El **dominio** de un atributo es el conjunto de los **valores permitidos** que puede tomar.
Formalmente, un **atributo** es una función que asigna un dominio al conjunto de entidades. Como un conjunto de entidades puede tener diferentes atributos, cada entidad se puede escribir como un conjunto de pares **(atributo, valor)**. Por ejemplo, un empleado se puede describir con el conjunto:

$$\{(DNI, 12345678), (Nombre, Alan), (Apellido, Gauss), (Ciudad, Rosario)\}$$

Tipos de atributos:

- **Simples**: no están divididos en subpartes (o sea no son compuestos).
- **Compuestos**: se pueden dividir en otros atributos (subpartes). Ejemplo: *nombre-persona* podría subdividirse en *nombre*, *primer-apellido*, *segundo-apellido*.

- **Monovalorados o monovaluados**: tienen un único valor para una entidad (o sea no son multivaluados).
- **Multivalorados o multivaluados**: tienen un conjunto de valores para una misma entidad. Ejemplo: un empleado puede tener cero, uno o más teléfonos, por lo que *numero-telefono* podría ser un atributo multivaluado.
- **Derivados**: Su valor se puede obtener a partir de valores de otros atributos. Su valor no se almacena, sino que se calcula cuando es necesario. Ejemplo: edad se puede obtener a partir de la fecha de nacimiento.

Obs: Un atributo toma un **valor nulo** cuando una entidad no tiene un valor para ese atributo.

2.3. Relaciones

Una **relación** es una asociación entre varias entidades. Formalmente, dados E_1, E_2, ..., E_n conjuntos de entidades, un **conjunto de relaciones** R es un subconjunto de

$$\{(e_1, e_2, ..., e_n) \mid e_1 \in E_1, e_2 \in E_2, ..., e_n \in E_n\}$$

donde una instancia (e_1, e_2, ..., e_n) es una instancia de la relación (de **grado n**).

Llamamos **relación binaria** a una relación de grado 2. No significa que la relación es entre dos entidades distintas ya que una entidad puede relacionarse consigo misma. Por ejemplo, la entidad *zona* puede relacionarse consigo misma con la relación *limita-con*.

Una relación puede tener atributos. Una muy común es *fecha* para distinguir entre las entradas que puede tener una relación entre las mismas dos entidades. Por ejemplo: la relación *préstamo* entre un *cliente* y un *libro* de una biblioteca podría tener el atributo *fecha_préstamo* para diferenciar las distintas veces en las que un mismo cliente se llevó el mismo libro.

2.4. Restricciones de asignación (mapping)

La **cardinalidad de asignación** expresa el **número de entidades** con las que se puede asociar otra entidad a través de un conjunto de relaciones.

En una relación binaria entre las entidades A y B, la cardinalidad debe ser:

$$\mathbf{1:1} \quad \mathbf{1:N} \quad \mathbf{N:1} \quad \mathbf{N:N}$$

La **cardinalidad** depende del mundo real que se está modelando. Por ejemplo, dadas las entidades *cuenta* y *cliente*, y la relación *CtaCli*, según las especificaciones de la empresa vamos a elegir una u otra cardinalidad:

- Si **una cuenta** puede pertenecer **únicamente a un cliente**, y **un cliente** puede tener **varias cuentas**, entonces la cardinalidad es **1:N** de *cliente* a *cuenta*.
- Si **una cuenta** puede pertenecer a **varios clientes** y **un cliente** puede tener **varias cuentas**, entonces la cardinalidad es **N:N**

Otra clase de restricción son las **dependecias de existencias**. Decimos que **x es dependiente por existencia de y** si la existencia de la entidad **x** depende de la existencia de la entidad **y**. Por lo tanto, si se suprime **y**, también se suprime **x**.

2.5. Claves en entidades

Definiciones sobre claves en entidades:

- **Superclave**: conjunto de uno o más atributos que permiten identificar de forma única a una entidad. Si K es una superclave, también lo será cualquier superconjunto de K.
- **Clave candidata**: superclave para la cual ningún subconjunto propio es superclave.
- **Clave primaria**: clave candidata elegida por el diseñador de la BD.
- **Entidad fuerte**: entidad que tiene una clave candidata.
- **Entidad débil**: entidad que no tiene suficientes atributos para formar una clave candidata y necesita de los atributos de otra/s entidad/es.
- **Discriminador**: conjunto de atributos de una **entidad débil** que, fijada una entidad fuerte, permite distinguir una entidad débil de otra. Por lo tanto, la **clave primaria** de una **entidad débil** está formada por su **discriminador** y la **clave primaria** de la entidad fuerte que depende.

2.6. Claves en relaciones

Sea R una relación que involucra a las n entidades $E_1, E_2, ..., E_n$, donde (E_i) es la clave primaria de E_i. La clave primaria de R depende de si tiene o no atributos propios:

- Si R **no tiene atributos**, entonces *atributo*(R) es **superclave** y tiene la forma:

$$\mathbf{atributo(R) = (E_1) \cup (E_2) \cup ... \cup (E_n)}$$

  Además, si la cardinalidad es **N:N**, entonces esta superclave es **clave primaria**. En cambio, si la cardinalidad es **N:1** o **1:N**, la clave primaria es un **subconjunto** de esta superclave.

- Si R **tiene atributos** $\{a_1, a_2, ..., a_m\}$, entonces *atributo*(R) es **superclave** y tiene la forma:

$$\mathbf{atributo(R) = (E_1) \cup (E_2) \cup ... \cup (E_n) \cup \{a_1, a_2, ..., a_m\}}$$

  Si hay atributos a_i que dependen de (E_i) entonces **no forman parte de la clave**.

2.7. Diagrama entidad-relación

Es la forma de representar el diseño de una BD. Consta de los siguientes componentes:

- **Rectángulos** $\rightarrow$ **entidades**.
- **Elipses** $\rightarrow$ **atributos**.
- **Rombos** $\rightarrow$ **relaciones**.
- **Líneas** $\rightarrow$ conecta los tres anteriores.

Hay tres formas distintas para escribir la cardinalidad de las relaciones, sin embargo probablemente la más cómoda es poniendo un **1** del lado de la **entidad** y **1** o **n** del lado de la relación. Por ejemplo: tenemos las entidades *dueño* y *propiedad*, entonces la cardinalidad de la relación *esDueño* depende de las especificaciones que tengamos:

1. **Un dueño** puede tener sólo **una propiedad** a su nombre y **una propiedad** puede pertenecer sólo a **un dueño**: la cardinalidad es **1:1** por lo que ponemos un **1** a cada lado de la relación.

2. **Un dueño** puede tener **varias propiedades** a su nombre y **una propiedad** puede pertenecer sólo a **un dueño**: la cardinalidad es **N:1** entonces ponemos **N** del lado de *dueño* y un **1** del lado de *propiedad*.

3. **Un dueño** puede tener sólo **una propiedad** a su nombre y **una propiedad** puede pertenecer a **varios dueños**: la cardinalidad es **1:N** entonces ponemos un **1** del lado del *dueño* y **N** del lado de *propiedad*.

4. **Un dueño** puede tener **varias propiedades** a su nombre y **una propiedad** puede pertenecer a **varios dueños**: la cardinalidad es **N:N** entonces ponemos **N** a cada lado de la relación.

Entonces la forma de pensarlo es pararse en una entidad, por ejemplo en *dueño*, y decir "*un dueño puede tener **N** (o **1**) propiedades, entonces va un **1** del lado de la entidad y **N** (o **1**) del lado de la relación*".

Una **entidad débil** se indica por medio de un **rectángulo de doble contorno**. Además la **relación** entre una **entidad débil** y la **entidad fuerte** de la que depende se considera una **relación débil** y se indica con un **rombo de doble contorno**.
Los **atributos** que forman parte de la **clave primaria** de una entidad o una relación se subrayan.

2.8. Reducción de DER a tablas (mapa canónico)

Una vez que hayamos determinado entidades, atributos y relaciones para una BD y hayamos armado el DER, podemos reducirlo a tablas. Esto es pasar cada entidad y cada relación a una tabla donde sus filas corresponden a los atributos.
Vamos a ver cómo pasar a tablas según si es una entidad fuerte, una entidad débil o una relación:

1. **Entidad fuerte**: Sea E una entidad fuerte con atributos $a_1, a_2, ..., a_n$. La tabla determinada por E tiene n filas, una por cada atributo. Los atributos que forman parte de la **clave primaria** deben estar **subrayados**.

2. **Entidad débil**: Sea A una **entidad débil** con atributos $a_1, a_2, ..., a_n$ y B la **entidad fuerte** de la que depende A con clave primaria $b_1, b_2, ..., b_m$. La tabla determinada por A tiene $n + m$ filas, es decir, el conjunto de atributos que quedan en su tabla es $\{\mathbf{a_1}, \mathbf{a_2}, ..., \mathbf{a_n}\} \cup \{\mathbf{b_1}, \mathbf{b_2}, ..., \mathbf{b_m}\}$. Deben quedar subrayados los atributos claves de $a_1, a_2, ..., a_n$ y todos los b_i.

3. **Relación**: Sea R una relación que involucra a las entidades $E_1, E_2, ..., E_m$, supongamos que $atributo(R)$ consta de n atributos. Entonces la tabla de R tiene n filas, una por cada elemento de $atributo(R)$. Recordemos que en $atributo(R)$ no sólo están los atributos propios de R sino que también la clave primaria de las entidades $E_1, E_2, ..., E_m$.

4. **Relación entre entidades fuertes y débiles**: Vemos este caso por separado porque es algo especial. Son relaciones **N:1** que **no tienen atributos propios** y la clave primaria de la entidad débil incluye la clave primaria de la entidad fuerte de la cual depende. Por eso estas relaciones son redundantes y no son necesarias pasar a tablas.

2.9. Generalizaciones y especializaciones

Una **generalización** es una **relación de inclusión** que existe entre una **entidad de nivel superior** y una o más (por lo general sólo 2) **entidades de nivel inferior**. Por ejemplo, podemos tener *cliente* y *trabajador* y para ambos tenemos atributos como fecha de nacimiento, sexo, nombre, DNI, etc., entonces los generalizamos en una entidad *persona* de forma que todos estos atributos van a esta nueva entidad y luego *cliente* puede tener atributos propios de un cliente como historial de transacciones, dinero en cuenta, etc. y *trabajador* puede tener atributos como ID, sector, puesto actual, sueldo, etc.
En el DER se representa mediante un **triángulo** etiquetado **IS_A**. Por lo general se pone el triángulo con un vértice hacia abajo, de modo que arriba queda la entidad más "general" (en este caso *persona*) y a cada lado de dicho vértice se ponen las entidades en las que se divide (*cliente* y *trabajador*).
Por otro lado tenemos la **especialización** que es la **relación inversa** de la generalización.

Al pasar una **generalización a tablas** existen dos métodos:

- **Crear una tabla para la entidad de nivel superior** como con cualquier entidad fuerte, y una tabla para cada entidad de nivel inferior con sus propios atributos más **los atributos de la clave primaria** de la entidad de nivel superior.

- **No crear una tabla para la entidad de nivel superior** y crear una tabla por cada entidad de nivel inferior de modo que incluya todos sus atributos más **todos los atributos** de la entidad superior.

3. Modelo Relacional

Ya vimos que el MER es una técnica de diseño de BD basada en objetos. Ahora lo que vamos a ver es el modelo relacional (MR) que es una formalización teórica de las bases de datos relacionales (BDR). Esta formalización nos va a servir luego para definir formas normales y demás.
Definimos una **BDR** como una **colección de tablas** donde cada una tiene un **nombre único**. En el MER hablábamos de tablas, filas, columnas, etc. para representar elementos de una BDR pero en MR los llamamos de otra manera. Algunas equivalencias entre MER y MR son:

- **Tabla** $\leftrightarrow$ **Relación**.
- **Fila** $\leftrightarrow$ **Tupla**.
- **Columna** $\leftrightarrow$ **Atributo**.

Definimos además, de forma similar que en MER, el **dominio** de un atributo como el conjunto de los posibles valores que puede tomar dicho atributo.
De este modo, una **relación** (tabla) con n atributos (columnas) es un subconjunto del producto cartesiano entre los n dominios.

Notación: Sea t una **tupla variable** de una relación R

- $\mathbf{t}$[**nombre_atributo**] indica el valor de t en el atributo *nombre_atributo*.
- $\mathbf{t \in R \equiv R(t)}$ indica que t está en R.

3.1. Esquemas e instancias

El **esquema** de una relación es su **diseño lógico**, es decir, su lista de atributos.
La **instancia** de una relación son los **datos** en un instante de tiempo dado.
Notación: Sea R una relación con atributos $a_1, a_2, ..., a_n$

- **esquema** $= (\mathbf{a_1}, \mathbf{a_2}, ..., \mathbf{a_n})$
- **instancia** $= (\mathbf{x_1}, \mathbf{x_2}, ..., \mathbf{x_n})$. Con $x_i \in Dom(a_i), \forall i = 1, ..., n$

El uso de **atributos comunes** en **esquemas de relaciones** es una forma de relacionar tuplas de distintas relaciones.

3.2. Claves

Las definiciones de superclave, clave candidata y clave primaria son las mismas que en MER, aunque a superclave la podemos redefinir formalmente de la siguiente forma:

Sea R una relación y r su esquema de relación. Un subconjunto K de r es una **superclave** si:

$$\forall t_1, t_2 \in R \ / \ t_1 \neq t_2 \Rightarrow t_1[K] \neq t_2[K]$$

Propiedades de relaciones en MR:

1. **No existen tuplas repetidas**: Los conjuntos por definición no incluyen elementos repetidos.
2. **Las tuplas no están ordenadas**: Los conjuntos no son ordenados, no tiene sentido referirse a la tuplas según su posición en la relación.
3. **Los atributos no están ordenados**: La cabecera de una relación se define como un conjunto de atributos. A diferencia de las columnas de una tabla que tienen un orden evidente, los atributos en las relaciones no lo tienen.
4. **Todos los valores de los atributos son atómicos**: Los dominios subyacentes son simples (no existen valores multivaluados). En cada posición de fila y columna dentro de una tabla, **siempre existe un sólo valor**, nunca una lista de valores.
5. Si una relación satisface estas condiciones, está **normalizada**. Esto se ve en profundidad en la sección (12).

4. DDLs, DMLs y SQL

4.1. ¿Qué son los DDL y DML?

Para poder operar con datos de manera eficiente y standard en el modelo relacional, existen los llamados Lenguajes de Definición de Datos (DDL por Data Definition Language) y Lenguajes de Manipulación de Datos (DML por Data Manipulation Language). En la teoría, estos son tipos de lenguajes distintos, aunque muchas veces son implementados por comodidad en uno solo, como veremos es el caso de SQL.

Los DDL son aquellos que definen la estructura de las relaciones a ser almacenadas. Es decir, con las sentencias en este lenguaje se definen las columnas que tendrá una tabla, los tipos de datos que le corresponden a cada una, si aceptan o no valores nulos, cuales forman parte de la clave, sus valores por defecto, etc. También es con esta herramienta que modificamos la estructura de una tabla o la eliminamos de la base de datos. Por último, con un DDL se crean los índices, en aquellos motores que soportan esta característica.

Por otro lado, en un DML es que llenamos con información lo definido con los anteriores. Con estos lenguajes insertamos filas a las tablas, las modificamos, eliminamos o incluso buscamos en las relaciones de la base. Aquí es donde entra la mayoría de la lógica a llevar a cabo al operar una BD. Encontrar el query que nos retorna exactamente lo que buscamos, de una manera eficiente, haciendo uso de las cualidades del motor, es el desafío de estos lenguajes.

4.2. ¿Y SQL?

El standard que implementa tanto un DDL como un DML es SQL (Structured Query Language). En él encontramos, además de lo mencionado en los párrafos anteriores, formas de definir vistas de tablas y herramientas para conceder o remover permisos para la seguridad de la BD (sección 9) y mecanismos para consultar información del sistema (sección 10).

La sección de SQL que corresponde a definición de datos está abarcada por los mecanismos `CREATE TABLE`, `DROP TABLE` y `ALTER TABLE` (las primeras dos tienen versiones equivalentes para vistas e índices, es decir, `CREATE VIEW` es por ejemplo también valido, pero con otros fines).

La sintaxis para crear tablas requiere de especificar un nombre a esta, uno a cada columna

junto con su tipo de datos asociado y seleccionar clave primaria, junto con las posibles claves foráneas. (Notar que SQL es bastante flexible con su sintaxis en varios aspectos, y por tanto esta no es la única forma)

```
CREATE TABLE nombre-tabla
  (definicion-columna [, definicion-columna], ...
  [definicion-clave-primaria]
  [,definicion-clave-ajena, ...]);
```

Aquí, una definición de columna consta de su nombre seguido de su tipo de dato, posiblemente seguido del literal `NOT NULL`, la clave primaria se indica como `PRIMARY KEY` (nombre-atributo) y la clave foránea indicando el nombre del atributo a la vez que la tabla y columna a la cual referencia.

Una tabla se puede modificar de 3 formas distintas, se crean columnas, se modifican o se eliminan. Para cada una de estos posibles tareas es que funciona `ALTER TABLE`, seguido de alguna de las palabras clave `ADD`, `MODIFY` y `DROP` respectivamente. Al añadir una columna, se especifica el nombre y el tipo (también se puede especificar si es clave primaria, etc.), junto con la posibilidad de seguir este comando por `BEFORE` y el nombre de alguna columna en la tabla. Esto ultimo tiene el efecto de introducir la nueva columna antes de la mencionada, de lo contrario y por defecto se añade al final. Al eliminar alguna, se especifica solo el nombre de aquella a ser eliminada. A la hora de modificar columnas, escribimos luego del comando el nombre de una columna existente con su nuevo tipo y posibles características llegado el caso de ser clave, no nulo, etc.

Por ultimo existen los casos en que se quiere eliminar una tabla. Esto es tan sencillo como el comando `DROP TABLE` nombre-tabla;

Algo importante a tener en cuenta es que SQL impedirá cualquier modificación que conlleve a algún problema en la BD. Por ejemplo, si el usuario no está autorizado para la operación, o si la eliminación de una tabla viola alguna de las leyes de integridad, tratadas en la sección (8).

Los tipos de datos que son soportados por el standard son:

Tipo de datos	Definición
CHAR(n)	Cadena de caracteres de longitud fija n
VARCHAR(n)	Cadena de caracteres de longitud variable máxima de n
INTEGER	Entero largo
DATE	Fecha
DECIMAL[(m[,n])]	Numero decimal de punto flotante de m bits, redondeado a n cifras decimales
SMALLINT	Entero corto (2 bytes)
FLOAT	Numero de punto flotante, equivalente a DECIMAL(16)
MONEY[(m)]	Decimal con 2 cifras decimales, equivale a DECIMAL(m,2)
SERIAL[(n)]	Entero único secuencial automático comenzando en n. Por omisión, n=1

Tabla 1: Tipos de datos en SQL

Cabe destacar que existen opciones de modificación de estos tipos de datos, como por ejemplo agregando la palabra UNSIGNED a los tipos numéricos. Para ver esto en mas detalle, conviene investigar en las opciones que provee cada motor.

Pasando a la parte de SQL que corresponde al DML, encontramos la posibilidad de buscar, insertar, modificar y eliminar. Esto se logra con SELECT, INSERT, UPDATE, y DELETE, respaldado por la capacidad expresiva que tiene SQL.

Primero abordamos SELECT, y con este sistema luego podemos entender más fácilmente los demás. En selección podemos especificar que columnas queremos, predicados que deben satisfacer, como agruparlos, incluso como ordenar el output. La estructura general es:

```
SELECT [DISTINCT] campo-1, ..., campo-n
FROM tabla-1, ..., tabla-m
[WHERE predicado-fila]
[GROUP BY campo(s)]
[HAVING predicado-grupo]
[ORDER BY campo(s) [ASC | DESC]];
```

Dentro de las secciones FROM, WHERE y HAVING se pueden a la vez realizar otras consultas, llamadas subconsultas. Esto permite realizar ciertas consultas de manera muy sencilla y muchas veces es preferible esto antes que juntar varias tablas mediante un producto cartesiano para reducir el uso de memoria.

En la parte SELECT se listan los campos a seleccionar del resultado final, notando que SQL no elimina repetidos por temas de eficiencia. En caso de ser esto deseable, se especifica la

palabra `UNIQUE`.
Luego de `FROM` se lista una secuencia de tablas a ser unidas por un producto cartesiano (ver sección 5). En caso de querer hacer un renombre, se escribe tabla-nombre AS nuevo-nombre o simplemente tabla-nombre nuevo-nombre, y luego los campos se acceden con nuevo-nombre.campo. En caso de no haber ambigüedades, se puede escribir directamente el nombre del campo.

Las condiciones de `WHERE` consisten principalmente de:

- Operadores proposicionales `AND`, `OR` y `NOT`
- Operadores de comparación (>, =, <>, etc.)
- `IS NULL` para chequear si un campo es nulo
- `IN` para chequear si una tupla se encuentra en otra tabla
- k `BETWEEN` n `AND` m para verificar si k se encuentra en el intervalo entre n y m
- `EXISTS` para verificar si una tabla es vacía o no
- `LIKE` para verificar si una cadena verifica una expresión regular

En `GROUP BY` se especifica una lista de campos por los cuales agrupar. Es decir, todas aquellas filas que sean iguales en estos campos conformaran una "nueva"tabla, y podrán ser operadas como tales, con el añadido de las llamadas .ªggregate functions". Estas son `COUNT`, `SUM`, `AVG`, `MAX` y `MIN`. Estas funciones aplicarán la operación dada por su nombre sobre las columnas de los distintos grupos.
En `HAVING` corresponden los predicados que filtran grupos, no filas, y se formulan de manera similar a los que aparecen en `WHERE`. Por último, `ORDER BY` indica como ordenar la salida, con la opción de ser ascendente o descendente. Tener en cuenta que esto siempre implica un trabajo extra al procesar la consulta.
Es importante resaltar que se debe respetar el orden dado para escribir estas consultas.

Además de esto, existen otros operadores para crear consultas, como `UNION`, `INTERSECT` y `EXCEPT` (resta de conjuntos) los cuales se escriben entre dos consultas para generar una nueva. Por lo general se omiten cuando sea posible, ya que son muy costosos.

Ahora, para modificar filas, escribimos

```
UPDATE tabla
SET campo=expresion, ...
[WHERE predicado]
```

para llevar a cabo la modificación de los campos sólo en aquellas filas que cumplen la condición, y similar para eliminar filas

```
DROP FROM tabla
[WHERE predicado]
```

Para insertar, escribimos `INSERT INTO` nombre-tabla `VALUES(valor-1, ..., valor-n)`. Opcionalmente, podemos especificar el nombre de la tabla y la columnas para las cuales proveeremos valores, rellenando las demás con sus valores por defecto o con valor nulo de no haberlo. Otra manera más avanzada de es insertar valores recuperados de un query hecha sobre otra tabla. En este caso, escribimos la subconsulta luego del nombre de la tabla. Por ejemplo,

```
INSERT INTO TEMP (P#, CANTTOTAL)
SELECT P#, SUM(CANT) FROM SP
GROUP BY P#;
```

Esto insertará todas las filas que resultan de la subconsulta en la tabla especificada.

4.3. Índices

Los índices son una herramienta que el motor de la BD utiliza para administrar la estructura de datos interna que utiliza para almacenar información. La creación de un índice sobre un atributo le dirá a este que será usual buscar por dicho atributo, y por lo tanto comenzará a almacenar cierta información para optimizar búsquedas sobre este atributo. Los índices por esta razón limitan en gran medida los accesos a disco, y como se verá en la sección (7), es por esto que la correcta utilización de índices impacta muy fuertemente en el rendimiento y eficiencia del motor. La incorrecta utilización de estos puede sobrecargar al motor de información inútil que no se corresponda con una mejora de rendimiento.

4.4. Otras funcionalidades

A la hora de utilizar SQL en una terminal, conviene conocer un par de comandos para hacer su uso más cómodo. Entre estos, están:

- `SHOW DATABASES`: Lista las bases de datos disponibles en el sistema.

- `USE` nombre-bd: Selecciona una base de datos de entre las disponibles.
- `SHOW TABLES`: Lista las tablas en la base de datos seleccionada.
- `DESC` nombre-tabla: Da la descripción de una tabla en una BD. Describe sus columnas y muestra la cantidad de filas en ella.
- `EXIT`: Útil para cuando apruebes la materia y no quieras saber mas nada de SQL.

5. Álgebra Relacional

Antes de hablar sobre el álgebra relacional, vamos a hacer una introducción de **lenguajes de manejo de datos (DMLs)**.
Un DML es una notación para expresar consultas, actualizaciones, inserciones y borrados de datos en BD. Un **lenguaje de consulta** es un lenguaje en el que un usuario solicita información de la BD.
Los lenguajes se clasifican en:

- **Algebráicos**: aplican operadores sobre relaciones (ISBL).
- **De cálculo**: especifican un predicado que las tuplas deben satisfacer. Pueden ser relacional de tuplas (QUEL) o relacional de dominios (QBE)
- **Intermedios**: SQUARE y SEQUEL (SQL).

Por otro lado pueden clasificarse en:

- **Procedimentales**: el usuario **da instrucciones** al sistema para que realice una secuencia de operaciones en la BD para calcular el resultado. → **álgebra relacional**.
- **No procedimentales**: el usuario **describe la información deseada** sin dar un procedimiento específico para obtener esa información. → **cálculo relacional**.

Es decir que el álgebra relacional es un DML procedimental. Consta de un conjunto de operaciones que toman una o dos relaciones como entrada y producen una nueva relación como resultado. Las operaciones fundamentales del AR son suficientes para expresar cualquier consulta.

5.1. Operaciones fundamentales

- **Selección**: operación **unaria** que selecciona tuplas de una relación r que satisfacen un predicado F dado:

$$\sigma_F(r)$$

- **Proyección**: operación **unaria** que devuelve sólo algunos atributos de una relación r:

$$\pi_{i_1,\ldots,i_k}(r)$$

  Se listan los atributos que se desea que aparezcan en el resultado con subíndices de π. Dado que el resultado es una relación, se eliminan todas las filas duplicadas.

- **Producto cartesiano**: operación **binaria** que permite combinar dos relaciones r y s:

  $$r \times s$$

  Se construye una tupla por cada par posible de tuplas. Si se tienen n_1 tuplas en r y n_2 tuplas en s entonces existen $n_1 \cdot n_2$ formas de elegir un par de tuplas, de forma que hay $n_1 \cdot n_2$ tuplas en $r \times s$.
  Definimos el **grado** de una relación como la cantidad de atributos. Luego, si $gr(r) = n$ y $gr(s) = m \Rightarrow gr(r \times s) = n + m$

- **Unión**: operación **binaria** cuyo resultado es una relación cuyo esquema es idéntico al de r o s y cuyo cuerpo está formado por todas las tuplas t pertenecientes a r, s o a las dos:

  $$r \cup s$$

  Para que sea válida, se exige que se cumplan dos condiciones: que $gr(r) = gr(s)$ y que los dominios del i-ésimo atributo de r y s sean iguales.

- **Diferencia**: operación **binaria** que permite encontrar tuplas que estén en una relación pero no en otra:

  $$r - s$$

  La relación resultante contiene aquellas tuplas que están en r pero no en s. Para que sea válida se exige que se cumplan dos condiciones: $gr(r) = gr(s)$ y los dominios del i-ésimo atributo de r y s deben ser iguales.

- **Renombre**: operación **unaria** que asigna un nuevo nombre x a una relación r:

  $$\rho_x(r)$$

  Para eliminar ambigüedades se nombran atributos mediante relación.atributo. Otra forma de ambigüedad surge cuando la misma relación aparece más de una vez en una consulta.

Una vez introducidos todos los operadores, podemos definir al **álgebra relacional (AR)** formalmente:

Una expresión general en el AR se construye a partir de subexpresiones. Sean E_1 y E_2 expresiones del AR. Entonces las siguientes son todas expresiones del AR:

- $E_1 \cup E_2$

- $E_1 - E_2$
- $E_1 \times E_2$
- $\sigma_P(E_1)$, donde P es predicado con atributos de E_1.
- $\pi_S(E_1)$, donde S es una lista de atributos de E_1.
- $\rho_x(E_1)$, donde x es el nuevo nombre de E_1.

5.2. Operaciones adicionales

Si nos restringimos sólo a estas operaciones algunas consultas son largas de expresar, por lo tanto, se definen operaciones adicionales que no añaden potencia pero simplifican consultas:

- **Intersección**: operador **binario** entre dos relaciones r y s cuyo resultado es una relación con cabecera idéntica a la de r o s y con cuerpo formado por tuplas t pertenecientes tanto a r como a s.

  $$\mathbf{r} \cap \mathbf{s}$$

  Para que sea válida se exige que se cumplan dos condiciones: $gr(r) = gr(s)$ y los dominios del i-ésimo atributo de r y s deben ser iguales.
  Su equivalente con operadores elementales es:

  $$\mathbf{r} \cap \mathbf{s} = \mathbf{r} - (\mathbf{r} - \mathbf{s})$$

- **Producto natural**: operador **binario** entre dos relaciones r y s:

  $$r \mid \times \mid s$$

  Equivale a realizar un $r \times s$, realizar una selección forzando la igualdad en atributos comunes, y finalmente, quitar las columnas duplicadas. Su equivalente con operadores elementales es:

  $$r \mid \times \mid s = \pi_{R \cup S}(\sigma_{R.A_1 = S.A_1 \wedge ... \wedge R.A_n = S.A_n}(r \times s))$$

  donde $r(R)$, $s(S)$ y $R \cap S = \{A_1, A_2, ..., A_n\}$ y el esquema resultante es $R \cup S$.

OBSERVACIÓN: Si $r(R)$ y $s(S)$ no tienen **ningún atributo en común**, entonces el producto natural de **r** con **s** es igual a su producto cartesiano. Es decir

$$R \cap S = \emptyset \Rightarrow \mathbf{r} \mid \times \mid \mathbf{s} = \mathbf{r} \times \mathbf{s}$$

- **División**: Sean los esquemas de relación $r(X_1, ..., X_m, Y_1, ..., Y_n)$ y $s(Y_1, ..., Y_n)$. Llamemos X al atributo compuesto $X_1, ..., X_m$ y Y al atributo compuesto $Y_1, ..., Y_n$.
  La división r/s es una relación con esquema X y cuerpo formado por todas las tuplas x tales que aparece una tupla (x, y) en r para todas las tuplas y de s.
  Formalmente, sean $r(R)$ y $s(S)$ relaciones y $S \subseteq R$

  - La relación r/s tiene esquema $R - S$.
  - Una tupla t está en r/s si para cada t_s en s existe t_r en r que satisface:

$$t_r[S] = t_s[S] \tag{1}$$

$$t_r[R - S] = t[R - S] \tag{2}$$

  Se establece para aquellas consultas que incluyen la frase "para todos".
  Su equivalente con operadores elementales es:

$$r/s = \pi_{R-S}(r) - \pi_{R-S}((\pi_{R-S}(r) \times s) - r)$$

  donde:

  - $\underline{\pi_{R-S}(r)}$ da todas las tuplas que satisfacen (2).
  - $\underline{\pi_{R-S}((\pi_{R-S}(r) \times s) - r)}$ elimina aquellas tuplas que no satisfagan (1).
  - $\underline{\pi_{R-S}(r) \times s}$ es una relación de esquema R que empareja cada tupla de $\pi_{R-S}(r)$ con cada tupla en s.
  - $\underline{(\pi_{R-S}(r) \times s) - r}$ da los pares de tuplas de $\pi_{R-S}(r)$ y s que no aparecen en r.
  - Si una tupla de t está en $\pi_{R-S}((\pi_{R-S}(r) \times s) - r)$ entonces existe alguna tupla en s que no se combina con la tupla t para formar una tupla en r.

  Por lo tanto t contiene un valor para los atributos $R - S$ que no tiene que aparecer en r/s. Estos son valores que se eliminan (al restar) de $\pi_{R-S}(r)$.

- **Asignación**: operador **unario** que permite asignar a una variable de relación temporal una expresión del AR.

$$var \leftarrow expresion$$

La evaluación de una asignación no da como resultado una relación que se presenta al usuario, su uso es únicamente para simplificar una expresión. El resultado de la expresión a la derecha de $\leftarrow$ es asignado a la variable de relación a la izquierda. Esta variable de relación puede usarse en subsiguientes expresiones.
Entonces puede reescribirse r/s como:

$$temp \leftarrow \pi_{R-S}(r)$$

$$temp - \pi_{R-S}((temp \times s) - r)$$

Definición:

Un lenguaje es **relacionalmente completo** si es al menos tan expresivo como el álgebra relacional.

6. Cálculo Relacional

En la materia se ven dos tipos de **cálculo relacional** distintos, uno es el de **tuplas** y otro el de **dominios**.

6.1. Cálculo relacional de tuplas

A diferencia del álgebra relacional, el **cálculo relacional de tuplas (CRT)** es un lenguaje de consulta no procedimental en el cual se describe la información deseada sin dar un procedimiento específico para obtenerla.
Una **consulta** en el CRT se expresa como

$$\{t/P(t)\}$$

Es decir, el conjunto de todas las variables de tuplas t tal que el predicado P (construido por átomos y operadores) es verdadero para t.
Ahora vamos a dar algunas definiciones formales del CRT:

- **Variable libre**: variable de tupla que no está cuantificada por un $\exists$ o un $\forall$.
- **Variable límite o acotada**: variable de tupla cuantificada por un $\exists$ o por un $\forall$.
- **Fórmula**: expresión en el CRT formada por átomos.
- **Átomo**: Puede tener una de las siguientes formas:
  1. $s \in r$, donde s es una variable de tupla y r una relación.
  2. $s[x]\ \alpha\ u[y]$ donde
     - s y u son variables de tuplas
     - x es un atributo sobre el que s está definida
     - y es un atributo sobre el que u está definida
     - α es un operador de comparación $(<, \leq, =, \neq, >, \geq)$
     - x e y deben tener dominios cuyos miembros puedan compararse.
  3. $s[x]\ \alpha\ c$ donde
     - s es una variable de tupla
     - x es un atributo sobre el que s está definida
     - α es un operador de comparación
     - c es una constante en el dominio de x.

Las fórmulas se construyen a partir de átomos usando las siguientes reglas (para nada vistas en otras materias):

1. Un átomo es una fórmula.
2. Si P_1 es una fórmula, entonces también lo son $\neg P_1$ y (P_1).
3. Si P_1 y P_2 son fórmulas, entonces también lo son $P_1 \vee P_2$, $P_1 \wedge P_2$, y $P_1 \Rightarrow P_2$.
4. Si $P_1(s)$ es una fórmula que contiene una variable de tupla libre s, entonces también son $\exists s \in r(P_1(s))$ y $\forall s \in r(P_1(s))$.

Además es posible escribir expresiones equivalentes como en AR. En CRT estas **equivalencias** incluyen tres reglas:

- $P_1 \wedge P_2$ es equivalente a $\neg(\neg P_1 \vee \neg P_2)$
- $\forall t \in r(P_1(t))$ es equivalente a $\neg\exists t \in r(\neg P_1(t))$
- $P_1 \Rightarrow P_2$ es equivalente a $\neg P_1 \vee P_2$

El CRT restringido a expresiones seguras es equivalente en poder expresivo al AR. Es decir, para cada expresión en el AR existe una expresión equivalente en el CRT y viceversa.

6.2. Cálculo relacional de dominios

A diferencia del CRT, el **cálculo relacional de dominios (CRD)** usa variables de dominio que toman valores del dominio de un atributo.
Una expresión en el CRD es de la forma

$$\{< x_1, x_2, ..., x_n > \mid P(x_1, x_2, ..., x_n)\}$$

donde $x_1, x_2, ..., x_n$ representan variables de dominio y P es una fórmula compuesta por átomos.
Un **átomo** en el CRD tiene una de las siguientes formas:

1. $< x_1, x_2, ..., x_n > \in r$ o $r(x_1, x_2, ..., x_n)$ donde
   - r es una relación en n atributos.
   - $x_1, x_2, ..., x_n$ son variables o constantes de dominio.
2. $x \, \alpha \, y$ donde

- x e y son variables de dominio.
- α es un operador de comparación.
- x e y tienen dominios que puedan compararse por medio de α

3. $x\,\alpha\,c$ donde
   - x es una variable de dominio.
   - α es un operador de comparación.
   - c es una constante en el dominio del atributo correspondiente.

Las **fórmulas** se construyen a partir de átomos usando las siguiente reglas (son análogas a CRT):

1. Un átomo es una fórmula.
2. Si P_1 es una fórmula, entonces también lo son $\neg P_1$ y (P_1).
3. Si P_1 y P_2 son fórmulas, entonces también lo son $P_1 \vee P_2$, $P_1 \wedge P_2$, y $P_1 \Rightarrow P_2$.
4. Si $P_1(x)$ es una fórmula en x, donde x es una variable de dominio, entonces también son fórmulas: $\exists x \in P_1(x)$ y $\forall x \in P_1(x)$.

6.3. Completitud relacional

En cuanto a poder expresivo, AR, CRT y CRD son equivalentes.
Habíamos dicho que un lenguajes es relacionalmente completo si es al menos tan expresivo como el álgebra relacional. Es decir si sus expresiones permiten la definición de cualquier relación que pueda definirse mediante expresiones de AR.
Luego, para demostrar que cualquier lenguaje L es completo basta con demostrar que L incluye análogos de cada una de las cinco operaciones algebraicas primitivas: selección, proyección, producto cartesiano, unión y resta del álgebra relacional.
Por ejemplo, SQL, QUEL, QBE son relacionalmente completos.

6.4. Comparación de lenguajes algebraicos y de cálculo

Por último vamos a ver una mini comparación entre los lenguajes que vimos.
Sabemos que el cálculo relacional es un lenguaje no procedimental, por lo que sus "consultas" especifican la información deseada pero no cómo obtenerla, a diferencia de AR que es procedimental y especifica el orden de las operaciones. Por eso se dice que los lenguajes

de cálculo son de más alto nivel que los algebraicos.
Veamos un ejemplo para compararlos:
Dadas las relaciones $R(A, B)$ y $S(B, C)$

1. La expresión algebraica:
$$\pi_C(\sigma_{A=a_1}(R \mid \times \mid S))$$
significa "listar los valores C asociados con el valor $A = a_1$ en la relación $\mid \times \mid$ de columnas ABC". Esta expresión da un orden particular de operaciones:
   1°) $R \mid \times \mid S$ ordena los valores B en ambas relaciones.
   2°) $\sigma_{A=a_1}$ selecciona las tuplas con $A = a_1$.
   3°) π_C muestra los valores C asociados.

2. La expresión algebraica (1.) es equivalente a
$$\pi_C(\pi_B(\sigma_{A=a_1}(R)) \mid \times \mid S)$$
para evaluar ésta realiza:
   1°) $\sigma_{A=a_1}(R)$ selecciona las tuplas de R con $A = a_1$.
   2°) π_B encuentra los B asociados.
   3°) $\mid \times \mid$ asocia las tuplas de S solamente para esos B.
   4°) π_C muestra los valores C asociados.

3. En el CRD:
$$\{c \mid \exists b : \ R(< a_1, b >) \wedge S(< b, c >)\}$$
expresa la información que se quiere pero no cómo obtenerla.

Observaciones:

- (1.), (2.) y (3.) son **equivalente**.
- Dependiendo de la organización de R y S, la opción (1.) puede llevar más tiempo que (2.).
- En base a esto se desprende el concepto de **optimización**, que consiste en convertir una expresión a otra equivalente de menor costo.

7. Optimización

En esta sección vamos hablar de optimización de consultas, pero antes de meternos de llenos con el tema arranquemos por preguntarnos ¿Qué es la optimización de consultas? La **optimización de consultas** es, dada una consulta, el proceso de elegir la forma más eficiente de evaluar las partes de la misma dentro de todas las estrategias posibles. Esto es de suma importancia cuando la consulta es compleja. Generalmente, el usuario al hacer consultas no elige la mejor estrategia, por lo cual es deber del sistema transformar estas consultas en formas equivalentes y mas eficientes.

En síntesis, el motor que procesa las consultas parsea el código SQL a una representación interna basada en álgebra relacional. Luego, esta expresión pasa por un programa llamado Optimizador, que es capaz de considerar cientos de estrategias diferentes y elegir la más eficiente para, finalmente, evaluar la expresión relacional dada.

También hay que considerar que este proceso de optimización puede ser parte de una actualización y no siempre se obtiene la forma más óptima, pero es una versión mejorada del esquema sin optimizar. Si bien este proceso mejora sustancialmente el costo de las consultas, normalmente es la velocidad de acceso al disco la que domina el costo.

7.1. Necesidad

Supongamos que estamos haciendo consultas sobre la querida y odiada tabla de Proveedores (S), Partes (P) y Proyectos (J) y queremos obtener los nombres de los proveedores que suministran la parte P2. Una posible formulación en SQL sería:

```
1 SELECT DISTINCT SNOMBRE
2 FROM S, SP
3 WHERE S.S# = SP.S# AND SP.P# = 'P2';
```

Ahora supongamos que la tabla S contiene 100 proveedores y que la tabla SP contiene 10.000 envíos de los cuales sólo 50 envíos son de P2.
Si el sistema realiza una consulta sin optimizar, la secuencia sería de la siguiente manera:

1. Calcular el producto cartesiano de las relaciones S y SP (S $\times$ SP).
   Esto es leer 100 veces c/u de las 10.000 tuplas de envíos, por lo que se leen 1.000.000 tuplas. Esta cantidad puede no entrar en memoria principal, lo cual produce un proceso de escritura en disco.

2. Restringir el resultado del paso 1 según el `WHERE`.
   Leer otra vez 1.000.000 de tuplas, para después obtener una relación formada por 50 tuplas que si caben en memoria principal.

3. Proyectar el resultado del paso 2 sobre `SNOMBRE`.
   Esto produce un máximo de 50 tuplas.

Este proceso puede producir mas de 3.000.000 accesos a disco!



Ahora vamos con un procedimiento equivalente pero más eficiente

1. Restringir la relación SP a las tuplas con el valor P2 en el atributo P#. Aquí se leen 10.000 tuplas, y el resultado consta solo de 50. Como son pocas, entran fácilmente en memoria.

2. Reunir el resultado del paso 1 con la relación S según S#. El paso 1 ya está en memoria, luego solo leemos 100 tuplas del disco, y el resultado contiene 50 tuplas que siguen en memoria principal.

3. Proyectar el resultado del paso 2 sobre `SNOMBRE`. Como está en memoria, no hacemos lectura del disco. Produce un máximo de 50 tuplas.

Este esquema produce 10.100 accesos a disco.

Ahora si tomamos el "número de operaciones de E/S de tuplas" como medida de desempeño tenemos que el segundo de estos procedimientos es unas 300 veces mejor que el primero. En el caso de que la tabla SP estuviera indexada según P#, la cantidad de operaciones de E/S del segundo procedimiento se vería reducida a 150, por lo que la mejora sería de 20.000 con respecto al primer esquema.

7.2. Proceso de optimización

El proceso de optimización consta de 4 partes esenciales. Primero la traducción de la consulta a una representación interna. Segundo (Francia) convertir la representación ob-

tenida a otra forma equivalente que mejore el rendimiento. Tercero es estimar el costo de las consultas. Cuarto, generar planes de consulta y elegir el más económico.

7.2.1. Traducción de la consulta a una representación interna

Para esto se utiliza el formalismo del álgebra relacional. Cada expresión del AR representa una secuencia de operaciones. Esta secuencia se puede representar como un árbol de consulta. Veamos un ejemplo

$$\pi_{\text{SNOMBRE}}(\sigma_{\text{P = 'P2'}}(S \ |\times|\ SP))$$

7.2.2. Conversión a una forma equivalente

Toda consulta puede escribirse de varias formas equivalentes. En el caso del ejemplo, la consulta tiene 8 formas distintas y equivalentes, por lo cual, este proceso trata de encontrar una representación más eficiente que la anterior. Para esto vamos a ver los siguientes criterios.

- Efectuar las selecciones antes que las reuniones.
  Reduce el tamaño de entrada y salida de la reunión (cantidad de datos a revisar y mostrar). Esto es muy importante ya que puede hacer la diferencia entre conservar esa salida en memoria principal o volcarla en disco. Ejemplos

$$\sigma_{\text{restricción-sobre-B}}(A \mid \times \mid B) = A \ |\times|\ \sigma_{\text{restricción-sobre-B}}(B)$$
$$\sigma_{\text{restricción-sobre-A} \,\wedge\, \text{restricción-sobre-B}}(A \ |\times|\ B) = \sigma_{\text{restricción-sobre-A}}(A) \ |\times|\ \sigma_{\text{restricción-sobre-B}}(B)$$

- Convertir condición de restricción a una condición equivalente en forma normal conjuntiva.
  Forma Normal Conjuntiva (FNC) es una condición formada por un conjunto de restricciones enlazadas mediante el operador lógico AND donde cada restricción se compone a su vez de un conjunto de comparaciones enlazadas sólo por el operador OR. Ejemplo tomando p, q y r como condiciones de restricción

$$\sigma_{p \vee (q \wedge r)}(r) = \sigma_{(p \vee q) \wedge (p \vee r)}(r)$$

  Las ventajas de la FNC son que la evaluación de condición resulta verdadera sólo si el resultado de evaluar cada conjunción es verdadero, lo que es equivalente: Resulta falsa si el resultado de evaluar cualquiera es falso. Si el resultado de evaluar una de

las condiciones es falso, no hace falta evaluar el resto y es trabajo del optimizador cual evaluar primero.

- Una secuencia de selecciones se puede combinar en una sola selección:

$$\sigma_{\text{restricción-2}}(\sigma_{\text{restricción-1}}(A)) = \sigma_{\text{restricción-2 AND restricción-1}}(A)$$

- Una secuencia de proyecciones se puede reducir a la última proyección aplicada:

$$\pi_{\text{proyección-2}}(\pi_{\text{proyección-1}}(A)) = \pi_{\text{proyección-2}}(A)$$

- Una selección de una proyección es equivalente a una proyección de una selección, siempre y cuando la restricción este basada en los atributos proyectados.

$$\sigma_{\text{restricción}}(\pi_{\text{proyección}}(A)) = \pi_{\text{proyección}}(\sigma_{\text{restricción}}(A))$$

- Considerar restricciones de integridad.
  El sistema sabe que SP.P# es una clave foránea que concuerda con la clave primaria P.P#. Ahora, la operación

$$\pi_{\text{S\#}}(SP \ |\times| \ P)$$

  representa los números de proveedores que suministran por lo menos una parte, pero dada las condiciones del sistema, se puede reducir a solamente

$$\pi_{\text{S\#}}(SP)$$

  con lo cual se elimina una reunión.

- Otras equivalencias.

$$\sigma_{\text{restricción-2}}(\sigma_{\text{restricción-1}}(A)) = \sigma_{\text{restricción-1}}(\sigma_{\text{restricción-2}}(A))$$

$$\sigma_{\text{restricción}}(A \cup B) = \sigma_{\text{restricción}}(A) \cup \sigma_{\text{restricción}}(B)$$

7.2.3. Estimación de costos de las consultas

Luego de convertir la consulta, el optimizador deberá evaluarla, considerando la distribución de los datos almacenados, la existencia de índices u otras rutas de acceso, el

agrupamiento físico de los registros, etc.

Existen distintos procedimientos de bajo nivel para realizar las operaciones de proyección, reunión y restricción. Por ejemplo, en el caso de selección, algunos de los procedimientos consideran si el campo de restricción está indexado o si la condición de igualdad es según una clave candidata o si los datos no están indexados pero están agrupados físicamente, etc.

Como cada procedimiento tiene un costo, el optimizador los elegirá valiéndose de la información en el catálogo:

- Nro. de tuplas de cada relación.
- Nro. de valores distintos de un atributo A en r.
- Tamaño del registro de una relación.
- Nro. de valores distintos de los datos en cada índice.
- Nro. de páginas ocupadas por cada relación.

entre otras más. Toda esta información que está en el catálogo, junto a métodos probabilísticos-estadísticos, permiten estimar el coste.

7.2.4. Generar planes de consulta y elegir el más económico

Este procedimiento consiste en generar un conjunto de planes, basados en el procedimiento anterior, y tomar el de menor coste. El costo en esencia es una estimación de las operaciones de E/S de disco requeridas.

En conclusión, generalmente tenemos que el tiempo de la optimización más el tiempo de la consulta optimizada es menor que el tiempo de la consulta sin optimizar. Donde podemos reducir el tiempo de optimización determinando una estrategia que es probable que tenga un coste bajo, analizando otras pocas y descartando otras sin completar el cálculo de coste.

8. Integridad

La **Integridad** de una base de datos se refiere a la corrección de la información contenida en ella. Decimos que la información es correcta cuando la base de datos es consistente.
Para asegurar la consistencia de los datos luego de cambios en la BD, se implementan **restricciones de integridad**. En pocas palabras, las restricciones de integridad son reglas que le marcan al sistema como actuar frente a distintos casos que podrían afectar la consistencia del sistema.
Distinguimos 3 tipos de restricciones de integridad: Restricciones de dominio, de las entidades y referenciales.

8.1. Restricciones de dominio

Por lo que entendimos de la sección (3) cada atributo está asociado a un dominio de posibles valores. La restricción de dominio se basa en limitar el dominio de nuestros atributos de modo que podemos dar un mejor comportamiento de ella frente a los posibles datos que se puedan introducir en nuestra BD.
Un claro ejemplo de limitación de dominio puede ser limitar atributos como el *peso* de una entidad a valores no negativos.
El SQL estándar soporta un conjunto restringido de tipos de dominio, estos son: Cadenas de caracteres de longitud fija, números en coma fija, enteros, enteros pequeños y fechas.
Las restricciones de dominio son **específicas**, ya que refieren a una BD en particular.

8.2. Restricciones de integridad de las entidades

Estas restricciones se pueden condensar en una simple regla, la **regla de integridad de las entidades**. Esta es:

"*Ningún componente de la clave primaria de la relación base puede aceptar nulos*"

- ¿O sea...?
- ¿De verdad lo tengo que explicar? Está bastante claro.
- Sí.
Bueno. Básicamente la regla nos habla de que como en el mundo real **todas** las entidades son distinguibles entre sí, y la **clave primaria** es una identificación única de cada entidad en nuestro modelo relacional, no podemos tener una entidad sin identificar, ya que sería una contradicción.

8.3. Restricciones de integridad referencial

Del mismo modo que las restricciones de entidad, las restricciones de integridad referencial se basan en una regla, la **regla de integridad referencial**, tal que:

"*La BD no debe contener valores de clave ajena sin concordancia*"

Esto sí que no se entiende. En otras palabras, la regla nos dice que si en una base de datos tenemos una **clave ajena** que no hace referencia a ningún valor, entonces esta clave ajena debe ser nula.
- Pero... ¿Qué es una clave ajena?
Una **clave ajena** (o clave foránea) es una atributo de una relación R cuyos valores concuerdan con los valores de una clave primaria en alguna otra relación R', donde R y R' no son necesariamente distintos.
Por ejemplo, Podemos tener la relación *depto*(nrodepto, presupuesto, ...) donde el atributo *nrodepto* es la clave primaria. Y tener la relación *empleado*(nombre, nroempleado, nrodepto, salario, ...) donde la clave primaria es *nroempleado*. Vemos que la relación *empleado* contiene un atributo (*nrodepto*) cuyos valores coinciden con los valores de la clave primaria de la relación *depto*, es decir, es una **clave ajena** en la relación *empleado*.

En el ejemplo anterior es fácil ver la regla de integridad referencial. Supongamos que tenemos la entrada de empleado:

$$(Mora, 018, Finanzas, ...)$$

pero en la relación *depto* no existe entrada cuya clave sea Finanzas. Esto implica que la entrada es contradictoria, para que esto tenga sentido, la única opción es que la entrada tenga un valor nulo en la clave ajena, tal que:

$$(Mora, 018, NULL, ...)$$

Es claro que una clave ajena y la clave primaria correspondiente deben definirse sobre el mismo dominio.
Una observación es que una relación podría incluir una clave ajena cuyos valores no nulos deben concordar con los valores de la clave primaria de esta misma relación. Por ejemplo podemos tener una relación:

$$empleado(\underline{nroempleado}, Nombre, Salario, nrogerente, ...)$$

Donde es claro que el empleado gerente no podrá tener un valor en el atributo *nrogerente*.

8.4. Como manejar las claves ajenas

Al momento de diseñar nuestro modelo debemos realizarnos 3 preguntas para cada clave ajena. Estas son:

1. *¿Puede aceptar nulos?*
   Esta respuesta depende de las políticas vigentes del mundo real representado en la base de datos. Puede haber atributos que tengan sentido que tengan valor nulo, como por ejemplo en la relación *empleado* pueden haber empleados que no tengan un departamento asignado de momento, luego la clave ajena tiene valor nulo.

2. *¿Que sucede si se intenta **eliminar** el elemento de una referencia de clave ajena?*
   Para responder esta pregunta, el diseñador puede tomar 3 acercamientos distintos. SQL ofrece comandos especializados para cada uno de estos comportamientos.

   - **RESTRINGIDA** (`RESTRICTED`):
     La operación de eliminación esta **restringida**. Esto significa que cuando se intente eliminar una entrada tal que se pierda la concordancia entre clave ajena y clave primaria la acción se rechazará.
   - **PROPAGACIÓN** (`CASCADES`):
     La operación de eliminación se **propaga** en cascada, eliminando también todos los elementos que tengan concordancia con el elemento eliminado.
   - **ANULACIÓN** (`NULLIFIES`):
     Se **asignan nulos** a la clave ajena en todos los elementos que tengan concordancia con el elemento a eliminar. Esto solo vale si la clave ajena acepta nulos.

3. *¿Que sucede si se intenta **modificar** la clave primaria del objetivo de referencia de una clave ajena?*
   La modificación funciona de manera análoga a la eliminación, ofreciendo 3 comportamientos diferentes. En SQL los comandos son los mismos.

   - **RESTRINGIDA** (`RESTRICTED`):
     La operación de modificación esta **restringida** al caso en el cual no existen elementos en concordancia. Si se intenta modificar un elemento que es objetivo de referencia de una clave ajena, esta acción se rechazará.
   - **PROPAGACIÓN** (`CASCADES`):
     La operación se **propaga** y modifica también la clave ajena de todos los elementos en concordancia.

- **ANULACIÓN** (`NULLIFIES`):
  Se **asignan nulos** a la clave ajena de todos los elementos correspondientes. Esto solo funciona con claves ajenas que acepten nulos.

Un ejemplo de la sintaxis de la creación de tablas con estas restricciones es:

```
CREATE TABLE tabla (
...
campos,
...
PRIMARY KEY pk,
FOREIGN KEY ak REFERENCES tabla-objetivo(nombre-atributo)
    NULLS [NOT] ALLOWED -- se aceptan (o no) nulos
    ON DELETE OF tabla-objetivo EFECTO
    ON UPDATE OF pk-de-objetivo EFECTO );
```

donde `EFECTO` = `RESTRICTED`, `CASCADES` o `NULLIFIES`.

9. Seguridad

Cuando hablamos del tema **seguridad** en la teoría de bases de datos hablamos de la protección de los datos contra una visualización, alteración o destrucción no autorizada. Esto implica controlar que los usuarios estén autorizados para llevar a cabo distintas acciones. El concepto de seguridad está fuertemente relacionado con el concepto de **integridad**, mientras que la seguridad asegura que los usuarios estén autorizados para hacer lo que tratan de hacer, la integridad asegura que lo que tratan de hacer sea correcto. (Para saber más sobre esto se puede consultar la sección dedicada a Integridad (8)).
Normalmente, el **administrador de la base de datos** (DBA) es el encargado de especificar restricciones, es decir, de decidir los diferentes derechos de acceso de los usuarios. Es claro que diferentes usuarios pueden tener diferentes derechos sobre un mismo objeto. En SQL contamos con dos herramientas independientes que nos ayudan a mantener la seguridad, estos son:

- El mecanismo de **vistas**.
- El subsistema de **autorización**.

Veamos estas herramientas más a fondo.

9.1. Vistas

En SQL, una tabla puede ser percibida por un usuario como una tabla base o una vista. ¿No sabes la diferencia? Yo te explico. Una **tabla base** es una tabla real, es decir que existe físicamente en algún disco, mientras que una **vista** es una tabla virtual que no existe por si misma, sino que se deriva de una o más tablas existentes. El usuario es ignorante a esto y cree que la vista existe de verdad.
La sintaxis para la creación de vistas en SQL es simple, mediante el comando `CREATE VIEW` tal que:

```
1 CREATE VIEW V
2 AS SELECT A, B, C
3    FROM R
4    WHERE P
```

Aquí se crea una vista V de la tabla R compuesta con los campos A, B y C que cumplen la condición P. Para el usuario es como si existiera una tabla llamada V.
Se dice que una vista es una ventana dinámica a través de la cual todas las modificaciones que la tabla real sufra se verán reflejados en esta.

Cualquier consulta a una vista se puede traducir en una consulta a la tabla base, simplemente se agregan las condiciones utilizadas para definir a la vista.
Un problema puede surgir cuando se utilizan las operaciones `UPDATE` y `INSERT` sobre una vista, debido a que esto puede causar que la condición que define a la vista no se cumpla para el elemento modificado o agregado. Para evitar esto se puede agregar una opción de verificación (`WITH CHECK OPTION`) a la consulta tal que verifique que toda fila modificada o insertada satisfaga la condición de la vista. De esta manera, la base de datos se modificará solamente si se cumple la condición de la vista.
Otro punto clave reside en la tenencia de la clave primaria. Una vista se puede actualizar solamente si ésta conserva la clave primaria. No es difícil ver por qué, si pensamos en modificar un elemento sin referirnos a su clave primaria, el sistema no sabrá identificarlo.

¿Por qué querríamos utilizar vistas?
Fácil, porque las vistas permiten a los usuarios ver solamente los datos que les interesan y son útiles, simplificando su trabajo y brindando seguridad al ocultar datos, dejándolos fuera de las vistas.

9.2. Subsistema de autorización

En SQL, el **subsistema de autorización** permite a los usuarios **conceder** de manera selectiva y dinámica derechos a otros usuarios y luego **revocarlos** si es necesario.
Los comandos pertinentes en SQL son:

- `GRANT` (otorgar).
- `REVOKE` (revocar).

El DBA tiene todos los derechos y puede concederlos a otros. Ahora imagínense el laburo del administrador en una red de cientos de usuarios, teniendo que conceder derechos para cada usuario, ¡es un montón!. Por suerte no lo tiene que hacer, SQL permite la opción de que un usuario al que se le haya sido concedido cierto derecho puede tener la habilidad de conceder ese derecho a otros usuarios.
La sintaxis general de este proceso se ve así:

```
GRANT comandos
  ON TABLE tablas
  TO usuarios
  WITH GRANT OPTION;
```

Algunos de los derechos aplicables sobre tablas y vistas son: `SELECT`,`UPDATE`,`DELETE`,`INSERT`.
Mientras que los derechos solo aplicables a tablas son: `ALTER TABLE` y `INDEX`.
La sintaxis básica de los comandos `GRANT` y `REVOKE` es:

```
1 GRANT SELECT ON TABLE S TO U4 WITH GRANT OPTION;
2 REVOKE SELECT ON TABLE S FROM U2;
```

Esto tiene pinta de descontrol, si todos se pueden dar derechos entre todos la seguridad de la tabla puede disminuir, por lo que cuando un usuario U1 le quita el derecho a otro usuario U2 (que le había sido otorgados por U1) este comando se propagará también hacia todos los usuarios a los que U2 les haya otorgado el derecho a eliminar.

10. Catálogo

El catálogo es una base de datos del sistema que contiene datos e información acerca de los objetos de interés para el sistema:

- Tablas
- Columnas
- Vistas
- Índices
- Usuarios
- Privilegios

entre otras.

10.1. Obtención de Información

El catálogo nos permiten obtener distintos tipos de información a partir de los datos que están guardados en el mismo, donde permite resolver el siguiente tipo de consultas:

- ¿Está autorizado un usuario? Privilegios de cada usuario.
- ¿Cómo se escribe el nombre de un campo?
- ¿Existe una determinada tabla?
- ¿Qué tipo de datos posee un determinado campo?
- ¿Existe algún índice para una determinada tabla? Si existe, el DBMS lo usa.
- ¿Cuantas filas tiene una determinada tabla?
- ¿Qué autorizaciones y password tiene un usuario? Si el usuario se olvida, el DBA lo puede buscar.
- ¿Quiénes tienen privilegios sobre una tabla dada?
- ¿Quién les dio esos derechos?

10.2. Características

El catálogo está formado por tablas dinámicas que el DBMS las mantiene. Estas se las puede relacionar (JOIN) y toda tabla tiene dueño. El dueño del catálogo es *System* y *System* otorga `GRANT SELECT TO PUBLIC`.

El DBMS lo usa para interpretar un comando y chequear autorizaciones. El optimizador usa información sobre los índices para elegir una estrategia de acceso posible. El subsistema de autorización utiliza información del catálogo referente a los usuarios y privilegios para conceder o rechazar solicitudes específicas de usuarios.

También hay que tener en cuenta que los catálogos no son idénticos, difieren en detalles.

10.3. Estructura y Contenido

- `SYSTABLES`: contiene una fila por cada tabla del sistema (tabla base o vista). Proporciona
  - Nombre de la tabla.
  - Usuario.
  - Cantidad de columnas.
- `SYSCOLUMNS`: contiene una fila por cada columna de cada tabla mencionada en `SYSTABLES`. Proporciona
  - Nombre de columna.
  - Nombre de la tabla de la cual forma parte.
  - Tipos de datos.
- `SYSINDEX`: contiene una fila por cada índice en el sistema. Proporciona
  - Nombre del índice.
  - Nombre de la tabla indexada.
  - Creador de la tabla.
- `SYSUSER`: contiene una fila por cada usuario en el sistema. Proporciona
  - Nombre.
  - Clave de acceso.
  - Nivel de privilegio.

- `SYSBAUTH`: contiene información sobre privilegios de acceso a nivel tabla. Proporciona
  - Propietario de la tabla.
  - Beneficiario del privilegio.
  - Nombre de la tabla.
  - Código de privilegio.
- `SYSCOLAUTH`: contiene una fila por cada privilegio columna atribuido a un usuario. Proporciona
  - Propietario de la tabla
  - Beneficiario del privilegio,
  - Nombre de la columna
- `SYSVIEWS`: Contiene sentencias de definición de vistas. Proporciona
  - `VIEW NAME`
  - `VIEW CREATOR`
  - `VIEW TEXT`
- `SYSSYNONYMS`: Contiene la lista de sinónimos creados por los usuarios. Proporciona
  - `USER-ID`
  - `CREATOR`
  - `TBNAME`
  - `ALTNAME`

  Los sinónimos se crean con:

```
CREATE SYNONYM C FOR CLIENTE;
```

10.4. Ejemplos

Veamos la estructura del catálogo para la base de datos de proveedores y partes

NAME	CREATOR	COLCOUNT	.....
S	José	4	.....
P	José	5	.....
SP	José	3	.....

Tabla 2: Ejemplo SYSTABLES

NAME	TBNAME	COLTYPE	.....
S#	S	CHAR	.....
SNOMBRE	S	CHAR	.....
SITUACION	S	CHAR	.....
.....	.....	.....	.....

Tabla 3: Ejemplo SYSCOLUMNS

NAME	TBNAME	CREATOR	.....
XS	S	José	.....
XP	P	José	.....
XSP	SP	José	.....
.....	.....	.....	.....

Tabla 4: Ejemplo SYSINDEXES

10.5. Consultas

Como son tablas igual que las ordinarias, se puede consultar mediante `SELECT`. Esto esta bueno por que un usuario no familiarizado con la base de datos puede entender su estructura mediante el catálogo. Ahora vamos a ver ejemplos de consultas al catálogo.

Averiguar cuáles tablas contienen una columna S#.

```
1    SELECT TBNAME
2    FROM SYSCOLUMNS
3    WHERE NAME = 'S#';
```

Obtenemos:

TBNAME
S
SP

Qué columnas tiene la tabla S.

```
1  SELECT NAME
2  FROM SYSCOLUMNS
3  WHERE TBNAME = 'S';
```

Resultado:

NAME
S#
SNOMBRE
SITUACION
CIUDAD

Cuantas tablas posee el usuario José.

```
1  SELECT COUNT(*)
2  FROM SYSTABLES
3  WHERE CREATOR = 'Jose';
```

10.6. Actualización

El catálogo no se puede actualizar utilizando `UPDATE`, `DELETE` o `INSERT` ya que esto sería peligroso, por lo que el sistema los rechaza. Las formas de actualización es mediante proposiciones de definición de datos, como `CREATE TABLE`, `CREATE INDEX`, etc.

Consideremos una tabla con n atributos, posibles ejemplos son:

- `CREATE TABLE`. Análogo a `INSERT`
  - Crea una fila en `SYSTABLES`
  - Crea n filas en `SYSCOLUMNS`
- `DROP TABLE`. Análogo a `DELETE`
  - Borra una fila en `SYSTABLES`
  - Borra n filas en `SYSCOLUMNS`
- `ALTER TABLE`. Análogo a `UPDATE`
  - Modifica una fila en `SYSTABLES`
  - Modifica n filas en `SYSCOLUMNS`

Luego con `UPDATE STATISTICS [ FOR TABLE nombre-tabla ];` tras varios `DELETE` o `INSERT`, se actualizan las tablas del catálogo que contienen información sobre cantidad de filas y tamaño de los índices de las tablas de las bases de datos.

11. Dependencias Funcionales

Las dependencias funcionales, abreviadas por DF a partir de ahora, son muy importantes casi no por si solas, sino por el peso que tienen en la teoría de la normalización de la sección (12). Las DF introducen una forma de relacionar atributos de tablas, lo cual provee más información que solo el esquema de dicha tabla, y son un tipo particular de restricción, visto en (8). Con las DF podemos redefinir el concepto de clave, se introduce la idea de descomponer relaciones mediante proyecciones, y todos los problemas que puede conllevar este proceso. Finalmente, las ideas y resultados de esta sección son ya suficientes para abordar la siguiente sección.

11.1. ¿Qué es una Dependencia Funcional?

Sea dada R una relación y α, β dos conjuntos de atributos de ella. Diremos que β **depende funcionalmente** de α, simbolizado por $\alpha \to \beta$, si y sólo si todo par de tuplas en R que coinciden en los campos α, entonces coinciden en los campos β. Lo escribimos de formalmente como

$$\alpha \to \beta \iff \forall t_1, t_2 \in R,\ t_1[\alpha] = t_2[\alpha] \to t_1[\beta] = t_2[\beta]$$

Si, por ejemplo, tenemos la relación R(PAÍS, CIUDAD, COD. POSTAL), es normal que todas las filas que coinciden en el código postal también coincidan en la ciudad, pero lo contrario no sucede (a modo de ejemplo, Córdoba, Argentina y Córdoba, España tienen códigos postales distintos). Luego, vale la dependencia COD. POSTAL $\to$ CIUDAD, PAÍS, pero no vale CIUDAD $\to$ COD. POSTAL.
Podemos por ejemplo agregar PAÍS al debate, y entonces un país y ciudad sí determinarían en este caso el código postal de la ultima, luego en este mismo ejemplo otra DF que es válida es PAÍS,CIUDAD $\to$ COD. POSTAL.

Volvamos al concepto de claves. Una **superclave** es un conjunto α de atributos tales que toda la tupla queda totalmente determinada por esta. Osea, buscamos en una superclave que $t_1[\alpha] = t_2[\alpha] \to t_1 = t_2$. Corroborando con la definición que dimos antes, decir esto es equivalente a decir que $\alpha \to R$. Tomaremos a partir de ahora esta definición para el concepto de superclave.

Este concepto se define como una condición más fuerte de lo que representa una DF. Decimos que β **depende funcionalmente de forma completa** de α si $\alpha \to \beta$ y además no existe $\gamma \subset \alpha$ tal que $\gamma \to \beta$. Es decir, si quitamos campos de α ya no tenemos la

información suficiente para determinar β.
Para complementar con la definición de superclave antes dada, las dependencias completas sirven para definir clave candidata. Un conjunto de atributos es **clave candidata** si el esquema entero de la relación depende de forma completa de tal. Esto es equivalente a la definición que ya conocíamos, la clave candidata determina de manera única una tupla, y además ningún subconjunto propio de ella lo puede hacer.
La importancia de las DF radica en que, al ser una generalización del concepto de clave y superclave, permiten expresar nuevas restricciones sobre los datos para modelar mejor los datos de la realidad.

11.2. Cierre y Axiomas de Armstrong

Tomemos el caso de una relación con atributos A,B,C, donde valen las DF A $\rightarrow$ B y B $\rightarrow$ C. Tomemos dos tuplas $t_1, t_2 \in R$ tales que t_1[A] = t_2[A]. La primera dependencia nos dice entonces que t_1[B] = t_2[B], y por la segunda dependencia tenemos que t_1[C] = t_2[C]. Es decir, hemos probado que A $\rightarrow$ C. Podemos decir entonces que la relación de dependencia funcional es transitiva.

Este resultado es muy importante, porque nos demuestra que podemos hallar nuevas dependencias funcionales. Cuando derivamos una DF de un conjunto F de dependencias, decimos que F implica lógicamente a tal DF. Definimos el **cierre de un conjunto de dependencias** F, F^+, como el conjunto de todas las dependencias implicadas lógicamente por F. Intuitivamente, este conjunto mantiene todas las relaciones deducibles partiendo con F como premisa. Diremos que dos conjuntos de DF son equivalentes si sus clausuras son iguales.

Existen 3 reglas, llamados Axiomas de Armstrong, que son reglas correctas y completas para deducir F^+. Es decir, con estas 3 reglas solo deduciremos DF que pertenecen al conjunto, y todas aquellas que pertenezcan pueden obtenerse mediante estos axiomas. Los axiomas son los siguientes

- Axioma de reflexividad: si $\beta \subset \alpha$, entonces $\alpha \rightarrow \beta$
- Axioma de aumento: si $\alpha \rightarrow \beta$, entonces $\gamma\alpha \rightarrow \gamma\beta$
- Axioma de transitividad: si $\alpha \rightarrow \beta$ y $\beta \rightarrow \gamma$, entonces $\alpha \rightarrow \gamma$

A partir de estos axiomas, se pueden probar 3 nuevas reglas, también útiles para calcular elementos del cierre

- Regla de unión: si $\alpha \to \beta$ y $\alpha \to \gamma$, entonces $\alpha \to \beta\gamma$
- Regla de descomposición: si $\alpha \to \beta\gamma$, entonces $\alpha \to \beta$ y $\alpha \to \gamma$
- Regla de pseudo-transitividad: si $\alpha \to \beta$ y $\gamma\beta \to \delta$, entonces $\gamma\alpha \to \delta$

Damos ahora una nueva definición, y es el **cierre de un conjunto de atributos** α. Lo denotamos también como α^+, y es igual al conjunto de todos los atributos implicados lógicamente por α. Como resultado inmediato, α es superclave de R si y sólo si $\alpha^+ = R$. Más allá de tener todas estas herramientas en el arsenal útiles para calcular el cierre de un conjunto de atributos, un algoritmo que realice este trabajo resulta muy útil para poder simplificar el trabajo. Así, si queremos calcular α^+, se puede seguir el siguiente procedimiento

```
resultado := α;
while cambios en resultado do
    for β → γ in F do
        if β ⊆ resultado then
            resultado = resultado ∪ γ;
        end
    end
end
```

11.3. Proyección y Preservación de Dependencias

Sea R una relación con F un conjunto de dependencias funcionales. Supongamos que R' es resultado de proyectar ciertos atributos de R. Osea, el esquema de R' está contenido en el de R. Definimos la **proyección** de F sobre R' como $\{\alpha \to \beta \in F \mid \alpha \subseteq R' \wedge \beta \subseteq R'\}$. Esta definición es muy natural, ya que contempla en R' solo las dependencias que comparten atributos con R'. Diremos que se **preservan dependencias** cuando, al descomponer una relación, el conjunto de todas las dependencias proyectadas sobre cada descomposición es equivalente al original. Es decir, en la descomposición aun se pueden expresar todas las dependencias. Este concepto se define más en profundidad en la sección de Normalización (12).

11.4. Recubrimiento canónico

Estas dos definiciones son necesarias para definir formalmente el concepto de recubrimiento canónico. La informalidad con que se define este tipo de recubrimientos debería

ser suficiente, además de que no usamos en la práctica este último concepto, por lo que realmente toda esta subsección puede omitirse.

Partimos de un conjunto F de dependencias funcionales. Diremos que A es **extraño** a α en $\alpha \to \beta$ si A $\in \alpha$ y F implica lógicamente a $(F - \{\alpha \to \beta\}) \cup \{\alpha - \text{A} \to \beta\}$. Es decir, los atributos extraños de α son aquellos que pueden removerse de tal y no afectar la clausura del conjunto.
De forma similar, B es extraño a β en $\alpha \to \beta$ si B $\in \beta$ y F implica lógicamente a $(F - \{\alpha \to \beta\}) \cup \{\alpha \to \beta - \text{B}\}$. Nuevamente, estos atributos son los que pueden removerse de β sin afectar la clausura.

El **recubrimiento canónico** de F, o también llamado minimal, consiste en un conjunto de DF F_c equivalente a F, donde las reglas cumplen ciertas reglas. Dicho conjunto nos da el conjunto de tablas normalizadas, cada dependencia correspondiendo a una de estas.
La definición formal de recubrimiento canónico de F es entonces un conjunto F_c tal que

- F implica lógicamente a F_c y viceversa
- Toda DF $\alpha \to \beta$ de F_c no tiene atributos extraños ni en α ni en β.
- Cada lado izquierdo de las DF en F_c es único, es decir, si $\alpha_1 \to \beta_1$ y $\alpha_2 \to \beta_2$ son dependencias distintas, entonces $\alpha_1 \neq \alpha_2$. Esto por lo general se puede resolver fácilmente usando la regla de unión para juntar reglas con su lado izquierdo repetido.

Otra definición posible dice que F_c es un recubrimiento canónico de F si

- Toda dependencia en este conjunto tiene un único atributo en el lado derecho
- No podemos quitar ninguna DF y seguir teniendo un conjunto equivalente
- No se puede reemplazar X $\to$ A por Y $\to$ A, con Y $\subset$ X, y que dicho conjunto siga siendo equivalente.

12. Normalización

Las Bases de Datos son de gran uso para cualquier sistema que dependa de la recolección o almacenado de grandes fuentes de información. Es importante, por ende, poder garantizar ciertos principios básicos, como el hecho de poder representar los datos lo más cercano a la realidad posible y no tener pérdidas de información no deseadas cuando algún dato es modificado o eliminado de la base, ni tampoco información repetida, por motivos obvios. Veremos que, con un mal diseño de BD, estos problemas ocurren y son imposibles de evitar. Es por eso que se presenta la **normalización**, una teoría acerca del modelado de BD que nos permitirá solucionar estos problemas.
La **Teoría de la Normalización** puede ser complementada con modelos de Entidad-Relación o usarse por si sola para hallar conjuntos de esquemas de relaciones que provean una buena representación de los datos a modelar.

12.1. Formas Normales y Dependencias Funcionales

Las **formas normales** son estados de las relaciones en donde se cumplen ciertos principios que definiremos más adelante. Cuando se cumplan estas condiciones, diremos que una relación esta en **primera**, **segunda** o **tercera forma normal**, abreviando cada una de ellas con 1NF, 2NF y 3NF respectivamente.
Una relación es aceptable y libre (de la mayoría) de errores que se encuentran en la practica si esta está por lo menos en 3NF. Existen formas más avanzadas todavía, como la 4NF, sin embargo aquí veremos hasta la considerada 3,5NF, mas conocida como **Forma Normal de Boyce-Codd**, o BCNF.
No todas las terceras formas normales son aceptables para una relación, ya que puede presentar ciertos problemas. Es por esto que el proceso práctico de normalización consiste, no sólo en llevar una tabla a alguna de sus 3NF, sino que además hay que hacer un análisis para decidir cual de todas estas formas es la que provee mas ventajas y menos perdidas.
Las dependencias funcionales (DF), explicadas en la sección (11), son la base teórica que necesitamos para la normalización. El solo análisis de las DF en una relación delatan, por lo general, de forma inmediata como se verá su 3NF. Las dependencias de una relación expresan como se relacionan sus campos, y se busca siempre poder aislar cada una de estas en una relación propia.

12.2. Mal Diseño de Bases de Datos

Los problemas que surgen con malos diseños se pueden reducir a tres:

- **Redundancia de Información**
  En un mal diseño ciertas dependencias entre campos se repetirán tantas veces como estos sean insertados. Esto significa uso excesivo de memoria y dificultad de para actualizar la información. Si una dependencia entre dos campos cambia, la actualización deberá llevarse a cabo en mas lugares haciéndola mas costosa, incluso posiblemente ocasionando problemas de integridad si en ciertos lugares la actualización se lleva a cabo y no en otros.

- **Imposibilidad de Representación**
  Si no se diseña bien una base de datos, habrá relaciones entre datos que no podremos representar, o que solo seremos capaz de hacerlo cuando seamos provistos con más información de la necesaria.

- **Perdida de Información**
  Similar al caso anterior, si se quieren eliminar datos de una relación, podríamos estar eliminando la ultima fila que posea la información que relaciona varios campos, y por tanto estaremos borrando información de más.

Estos 3 problemas están asociados a las operaciones `UPDATE`, `INSERT`, `DELETE` respectivamente. La solución a todos estos problemas se obtiene al normalizar una base de datos, por lo menos hasta su 3NF.

Si la explicación anterior no quedó clara no importa, porque una imagen vale mas que mil palabras

S#	CIUDAD	P#	CANTIDAD
S1	Rosario	P1	300
S1	Rosario	P3	400
S1	Rosario	P4	200
S1	Rosario	P2	200
S1	Rosario	P5	100
S1	Rosario	P6	100
S2	Concordia	P1	300

Tabla 5: Ejemplo con los 3 problemas de diseño

En la tabla (5) se pueden observar los 3 problemas antes mencionados.
Primer problema, hay información redundante. La relación del proveedor con su ciudad se repite muchas veces, tantas como la cantidad de veces que aparezca el primero. Si

quisiéramos actualizar la ciudad de S1 tendríamos que modificar las primeras 5 filas. Si S1 proveyese un millón de partes, la actualización debe darse en ese millón de filas. Otro problema es ¿qué ocurriría si por una falla, las primeras tres se actualizan, y las otros dos no? ¿Cuál es realmente la ciudad del proveedor S1? En general, seria muy complicado saberlo.

Segundo problema, se nos hace saber que el proveedor S3 estará situado en Córdoba, pero todavía no provee ninguna pieza. Esta información de momento no puede ser representada. No es posible insertar la tupla (S3, Córdoba, NULL, NULL), ya que P# es clave, luego no puede ser NULL. Esto significa que no podemos agregar la relación entre S3 y Córdoba hasta que S3 no provea alguna parte. Esto, claramente, no es deseable.

Tercer problema, si S2 ya no provee P1, entonces lo deseable es eliminar dicha tupla de la relación. Ahora, hacer esto supone perder la información de que S2 está situado en Concordia. Osea que perdemos más información al eliminar datos de la tabla.

De aquí en adelante consideraremos la relación R con esquema (S#, CIUDAD, P#, CANTIDAD, SITUACIÓN), suponiendo que se cumplen las

$$\text{DF} = \Big\{\text{S\#} \to \text{SITUACIÓN, SITUACIÓN} \to \text{CIUDAD, S\#,P\#} \to \text{CANTIDAD}\Big\}$$

como ejemplo para el proceso de normalización.

12.3. Primera Forma Normal

*Una relación está en **1NF** si y sólo si todos los dominios simples subyacentes contienen sólo valores atómicos.*

Un dominio simple subyacente es en pocas palabras el dominio de un campo en una relación. Lo que se pide para la primera forma normal entonces es que los campos almacenen valores atómicos. Mas sencillo todavía, la 1NF nos pide que en toda fila de toda columna haya un único valor.

OFICINA	SECRETARIA	TELÉFONOS
O1	Ana	15 12-3456, 15 12-9932
O2	Eugenia	15 13-7632

Tabla 6: Tabla no en 1NF

OFICINA	SECRETARIA	TELÉFONO
O1	Ana	15 12-3456
O1	Ana	15 12-9932
O2	Eugenia	15 13-7632

Tabla 7: Tabla en 1NF

La conversión a 1NF es directa, y por lo general siempre partimos de una relación que cumple con esta.
Observación: Al convertir la tabla a una 1NF a veces será necesario actualizar la clave, ya que ahora hay nuevas filas que comparten varios campos.

Al pasar la relación R a una 1NF, llegamos a una relación donde las dependencias son las del siguiente diagrama:

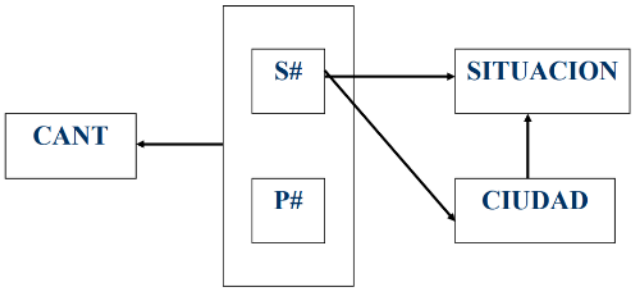


Figura 1: Diagrama de dependencias funcionales de R

12.4. Segunda Forma Normal

Un atributo **primo** es un atributo que forma parte de la clave primaria.

> *Una relación R está **2NF** si y sólo si está en **1NF** y cada uno de sus **atributos no primos es dependiente funcional completo de cada clave candidata** de R.*

Lo que nos quiere decir esta definición es que una relación en 2NF, además de estar en 1NF, cumple con el hecho de que todo atributo que no forma parte de la clave primaria entonces esta determinado por la clave entera, y no por un subconjunto de esta.

Para el caso de R esto no vale, porque la clave es S#,P#, pero CIUDAD y SITUACIÓN solo dependen de S# y no de ambas. Luego, separamos R en dos relaciones, SP(S#,P#, CANTIDAD) y R'(S#, SITUACIÓN, CIUDAD). Las dependencias son entonces como en las siguientes imágenes



(a) Dependencias funcionales en R' (b) Dependencias funcionales en SP

Figura 2: Conjunto de relaciones en 2NF equivalentes a R

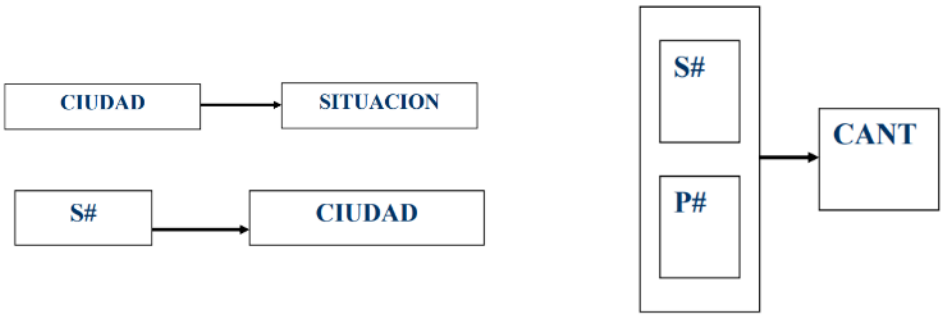
A modo de conclusión, la segunda forma normal elimina dependencias incompletas de la clave primaria, subdividiendo esta clave en otras relaciones si hace falta. Esto, en gran medida, solucionará problemas de redundancia y demás, pero no los resuelve a todos.

12.5. Tercera Forma Normal

Una relación está en 3NF si y sólo si está en ***2NF*** *y* ***todos los atributos no primos dependen de forma no transitiva de la clave primaria.***

Osea, estamos pidiendo ahora que los atributos no primos no solo dependan en forma completa de la clave primaria, sino que solo lo hagan de esta. En el caso de la relación R' antes derivada esto no vale, porque CIUDAD no depende únicamente de la clave S#, sino que también lo hace de SITUACIÓN. Luego R' no es 3NF, pero SP si lo es.

El proceso de normalizar a 3NF por ende consiste de llevar primero la relación a un conjunto de relaciones 2NF, y luego eliminar las dependencias transitivas, si las hay. Con nuestro ejemplo, proyectamos los atributos S#, SITUACIÓN a una tabla S(S#,SITUACIÓN), y SITUACIÓN, CIUDAD a otra T(SITUACIÓN,CIUDAD).



12.6. Pérdida de Información

Si bien el requerimiento mínimo para una base de datos es que esta se encuentre en alguna tercera forma normal, no cualquiera es aceptable. Esto se debe a que, en una mala descomposición en el proceso de normalización puede llevar a pérdidas de información irrecuperables.

Cuando normalizamos una tabla, la operación asociada del álgebra relacional que utilizamos es la proyección. Creamos nuevas tablas proyectando campos de la tabla a normalizar. En algún momento, requeriremos juntar esta información de nuevo, es decir, nos interesará reunir las nuevas relaciones para obtener la original y poder realizar las mismas consultas que podíamos hacer antes. Este proceso reverso lo logramos con el producto natural. El problema ocurre cuando este proceso inverso no se puede realizar porque la descomposición así no lo permite.

Cuando sucede perdida de información, al hacer el producto natural entre dos relaciones obtenemos una tabla con más filas de las que en realidad deberíamos tener. Como no sabemos cuales corresponden estar y cuales no es que perdemos información, al no poder discriminar entre filas correctas y filas creadas por error.

Para asegurar que no este problema no existirá, hacemos uso de un resultado que establece que, si $\alpha \cap \beta \rightarrow \alpha$ o $\alpha \cap \beta \rightarrow \beta$ están F^+, entonces $\alpha \,|\times|\, \beta$ no presenta perdidas. Cuando se lleva a cabo el proceso de normalización verificaremos de esta forma que es en efecto valida la descomposición hecha.

12.7. Pérdida de Dependencias Funcionales

Otro posible problema que se suele encontrar al normalizar relaciones es la pérdida de dependencias. Este inconveniente, no tan grave como el anterior, consiste en que algunas dependencias funcionales ya no pueden ser expresadas en la descomposición final.
Si se toma como ejemplo el esquema R(A, B, C) con el conjunto de dependencias $F = \{A \to B, B \to C\}$ y descomponemos en R_1(A,B), R_2(A,C), entonces ya no podemos expresar en estas relaciones la dependencia $B \to C$.

Este problema no es tan grave, pero si existe una tercera forma normal que nos evite perder dependencias entonces esta será preferible.

Para determinar si una descomposición $R' = \{R_1, \ldots, R_n\}$ de R con dependencias funcionales F preserva DF, queremos que $F^+ = F'^+$, donde F' son las dependencias de F que se mantienen en la descomposición R'. En términos mas precisos, $F' = \bigcup_{i=1}^{n} F_i$ y F_i es la proyección de F sobre R_i. Esta igualdad de conjuntos de dependencias expresa que todas las relaciones dadas por DF de R pueden ser expresadas en la descomposición R'.

Si $F = F'$, entonces trivialmente vale $F^+ = F'^+$ por lo que no perdemos dependencias. Sin embargo, no vale el recíproco, luego $F \neq F'$ no implica perdidas de DF. En estos casos, puede ocurrir que una dependencia F_1 implique otra F_2, luego perder F_2 en F' no es un problema ya que $F_2 \in F'^+$ de igual manera.

12.8. Forma Normal de Boyce-Codd

Edgar F. Codd desarrolló las tres primeras formas normales que hemos visto. Sin embargo, existen casos particulares en donde incluso la 3NF puede presentar problemas. Raymond F. Boyce propuso como mejora a la 3NF lo que hoy se conoce como la Boyce-Codd Normal Form.

Una relación R está en BCNF si y sólo si R está en 3NF y ***todo determinante de las dependencias funcionales en R son clave candidata de esta.***

En realidad, basta con chequear que vale esta nueva condición para poder afirmar que la relación ya se encuentra en 3NF.

El proceso de normalización para una BCNF no lo cubrimos porque en la practica de la materia no se aplica, pero es importante entender cuando una relación tiene esta forma.

La idea fundamental es, nuevamente, proyectar una relación en un conjunto de relaciones tales que en el resultado final, los determinantes de las DF proyectadas sobre cada descomposición obtenida cumpla con ser clave candidata de tal.
Un método sencillo para determinar si una relación en 3NF está en BCNF es analizar si tiene claves candidatas que se solapen. Si no existen dos claves candidatas tales que compartan algún campo, entonces dicha relación está garantizado de estar en forma normal de Boyce-Codd.
Es importante también tener en mente que a veces, una relación no podrá llevarse a una BCNF. Incluso habrá veces donde exista una descomposición que si lo esté, pero que, por ejemplo, pierda dependencias funcionales. Cuando se da este caso, lo preferible es no perder dependencias funcionales, aun si esto significa no tener una BCNF.

13. Recuperación

La **recuperación de transacciones** está vinculada al concepto de procesamiento de transacciones.
Pero, ¿Qué es una transacción? Una **transacción** se define como la unidad lógica de trabajo. Se puede pensar como qué acción quiero realizar sobre una determinada base de datos, sin considerar los pasos intermedios que tengo que realizar para lograrlo. Por ejemplo, si tenemos una tabla P en donde existe un campo R que es la suma de todos los valores de otro campo A. Una transacción puede ser agregar un nuevo elemento a la tabla, pero si agregamos un nuevo elemento con un nuevo campo A, luego debemos incrementar el campo R, por lo que la transacción está compuesta por un secuencia de dos operaciones, `INSERT` para agregar el nuevo elemento, y `UPDATE` para actualizar el campo R.
Habiendo entendido esto, podemos dar la definición formal de **unidad lógica de trabajo**. Esta dice:

*"Una unidad lógica de trabajo es una secuencia de operaciones con la cual un **estado consistente** de la base de datos se transforma en otro estado consistente sin conservar por fuerza la consistencia en todos los puntos intermedios."*

- ... ¿Qué?
Básicamente, se asegura que todas las modificaciones que la transacción realizará se ejecutarán todas y al terminar, el estado nuevo de la base de datos será un estado consistente (tendrá sentido).
Puede pasar que durante la ejecución de una de las modificaciones del medio se produzca un error y el estado sea **inconsistente**, en este caso la transacción se anula y se vuelve al estado original, es decir, a un estado consistente. Se puede decir entonces que una transacción se lleva a cabo en su totalidad o se cancela en su totalidad.
Puede pensarse este concepto de unidad lógica de trabajo como una **operación atómica** que, o bien se realiza por completo, o no se realiza, sin presentar estados intermedios.

13.1. Manejador de transacciones

El componente del sistema encargado de lograr esa atomicidad se conoce como el **manejador de transacciones** y provee dos operaciones claves:

- `COMMIT` (comprometer):
  La operación `COMMIT` (o comprometerse) señala que una transacción a terminado de manera exitosa y que la base de datos está nuevamente en un estado consistente y le dice al manejador que todas las modificaciones efectuadas se pueden **hacer permanentes**.

- `ROLLBACK` (retroceder):
  La operación `ROLLBACK` (o retroceder) marca que durante la transacción algo salió mal y no termino exitosamente. Avisa al manejador de transacciones que la base de datos podría estar en un estado inconsistente y que todas las actualizaciones realizadas deben anularse.
  Para lograr volver a un estado anterior, el sistema mantiene una bitácora con los detalles de las operaciones de actualización con los valores iniciales y finales del objeto modificado.

13.2. Punto de sincronización

Un **punto de sincronización** representa un límite entre dos transacciones consecutivas, de modo que corresponde al final de una unidad de trabajo, y es el punto en el cual una base de datos debería estar en un estado consistente.
Los momentos de un programa que establecen un punto de sincronización son:

- Cuando se realiza un `COMMIT`.
- Cuando se realiza un `ROLLBACK`.
- Al comienzo del programa.

Cuando se llega a un punto de sincronización, dependiendo si se llega con un `COMMIT` o con un `ROLLBACK` se comprometerán o se anularán las modificaciones realizadas desde el punto de sincronización anterior. Se cierran todos los cursores abiertos y se pierde todo posicionamiento en la base de datos. Además se liberaran todos los registros que estén siendo bloqueados (para saber mas sobre bloqueos leer la sección 14).
Como cada `COMMIT` o `ROLLBACK` termina una transacción e inicia la siguiente, se dice que las transacciones no solo son la unidad de trabajo sino que también son la **unidad de recuperación**.

13.3. Procotolo de bitácora de escritura adelantada y Punto de Revisión

Para mantener la traza de los estados de la base de datos, y que ésta sobreviva aún si el sistema se cae en el instante siguiente, se debe escribir físicamente antes de poder completar el procesamiento de una instrucción `COMMIT`. Cuando no se logra completar exitosamente una transacción, el procedimiento de reinicio recuperará las modificaciones

que hayan sido escritas en el disco, por lo que, las que no fueron grabadas, se pierden.

Así mismo, el sistema establece cada cierto intervalo de tiempo **puntos de revisión**. Llegado este punto, el sistema graba de forma permanente las transacciones finalizadas, moviéndolas de los buffers al disco, y además se guarda un **registro de punto de revisión** en la bitácora, con información necesaria para el proceso como lo viene a ser una lista de las transacciones en curso. Con este ultimo dato, tras una falla del sistema se busca el punto de verificación más reciente y con este se determina qué acciones tomar con cada transacción. Llamamos t_f al instante de fallo y t_r al instante de la ultima revisión.

- Si la transacción comenzó y terminó antes de t_r, entonces ya se encuentra escrita de forma permanente.
- Si la transacción finalizó después de t_r entonces debe realizarse otra vez, ya que estos cambios se perdieron.
- Si la transacción no llegó a finalizar antes de t_f, entonces se realiza un retroceso, es decir, se comienza de nuevo la transacción.

13.4. Fallas

Un sistema puede sufrir distintos tipos de fallas, provenientes de diversas fuentes. Estas fallas pueden ocurrir en distintos scopes, estos son:

- Fallas **locales**, que afectan solamente a las transacciones.
- Fallas **globales**, estas tienen que ver con los sistemas de bases de datos distribuidos, es decir sistemas compuestos por varias bases de datos. Esto implica que una falla global pueda afectar a varias o todas las bases de datos involucradas.

De este modo dividimos las fallas en varios tipos:

- **Fallas del sistema**:
  Afectan a todas las transacciones que se están realizando, pero **no dañan** físicamente a las base de datos. Por ejemplo, una interrupción de la electricidad.
  En estos casos, se **pierde el contenido** de la memoria principal, por lo que se pierde el estado preciso de la transacción que se realizaba en el momento de la falla.

- **Fallas en los medios de almacenamiento**:
  Estas **dañan físicamente** a la base de datos, como puede ser una rotura del disco.

Esto implica cargar de nuevo o restaurar la base de datos a partir de un respaldo y utilizar la bitácora para realizar todas las transacciones nuevamente desde el momento que se realizó la copia de respaldo.
Este tipo de fallos puede ser irrecuperable si no existe un duplicado de la información. En Sistemas de Bases de Datos Distribuidos estos problemas pueden ser mitigados al no centralizar los datos y no tener que depender exclusivamente de una pieza de hardware.

14. Concurrencia

"Hay que usar locks (TRABAS) para evitar deadlocks (ENLACES MORTALES)."

- Mateo Bacci (2023)

En lo que a bases de datos se refiere, dos transacciones son **concurrentes** cuando estas necesitan acceder a la misma base de datos al **mismo tiempo**.

14.1. Problemas de concurrencia

La falta de sincronización puede derivar en problemas de concurrencia, donde las transacciones escriban y lean datos al mismo tiempo, causando que una transacción aparentemente correcta produzca un resultado incorrecto.
Estos problemas se pueden clasificar en 3 problemas distintos:

- **Modificación perdida**
  El problema de la modificación perdida es el mas simple de los problemas de concurrencia, y se trata del caso en el cual **dos transacciones leen y actualizan el mismo dato sin ver lo que la otra había actualizado**, logrando que se pisen las modificaciones y se pierdan.
  Por ejemplo, sea el dato R y las transacciones A y B. Si no hay sincronización, se puede dar el siguiente orden de eventos: Primero, A lee R, luego B lee R. Ahora, A actualiza R y escribe el valor actualizado del mismo. Por último, B actualiza R y escribe este valor. Veamos que B escribió el valor actualizado de R sin haber tomado en consideración lo que había escrito A, y este se perdió.

- **Dependencia no comprometida**
  Para explicar este error primero debemos recordar como funciona una transacción. Luego de leer o modificar un dato, la transacción debe **comprometerse** (commit), es decir, debe confirmar que los cambios realizados serán los que se escribirán. Es como una especie de ¿Estás seguro? Si una transacción no se compromete por algún error, vuelve el estado atrás y deja todo como estaba antes de ejecutarse, se dice que **retrocede** (rollback). Para saber mas sobre transacciónes consultar la sección (13).
  Ahora podemos empezar a hablar del problema de la dependencia no comprometida. Este problema se da cuando **una transacción A trabaja sobre datos que no han sido comprometidos todavía por otra transacción B** (se podría decir que trabaja sobre "especulaciones"). Si la transacción B se compromete, entonces

no hay problema, sin embargo si por alguna razón esta retrocede, entonces todo el trabajo que A esta haciendo es incorrecto.
Un peor caso es cuando A actualiza una modificación no comprometida y pierde esa actualización por un retroceso.

- **Análisis inconsistente**
  El problema del análisis inconsistente se produce cuando **durante la ejecución de una transacción A, uno de los datos ya leídos es modificado por otra transacción B** (y comprometido, por lo que no hay dependencia no comprometida). Esto produce que el estado de la base que A está usando es **inconsistente** con el estado actual, y esto genera que A realice un **análisis inconsistente**.

14.2. Bloqueos

Para asegurar que los objetos de nuestras transacciones no cambien de manera no predecible durante su ejecución utilizamos **bloqueos** (locks (trabas)). Éstos bloquean el acceso de otras transacciones a los datos que están siendo utilizados y evita que sean modificados. Existen dos tipos de bloqueos:

- **Bloqueo exclusivo** (X):
  Si una transacción A tiene un bloqueo exclusivo sobre un dato R provoca que si cualquier otra transacción B intenta bloquear a R, esta se quedará en un estado de espera hasta que la transacción bloqueante libere el bloqueo.

- **Bloqueo compartido** (S):
  Si una transacción A tiene un bloqueo compartido sobre un dato R entonces, si otra transacción B intenta bloquear de manera exclusiva a R, esta quedará en un estado de espera hasta que el bloqueo se levante. Sin embargo, si el bloqueo que se intenta hacer sobre R es un bloqueo compartido, B podrá acceder a R y realizar su bloqueo. Es decir, R estará bloqueado tanto por A como por B.

Como usuarios, la solicitud de realizar bloqueos sobre datos está implícita en las transacciones que realizamos. Por ejemplo, leer un registro implica adquirir un bloqueo compartido sobre el registro, mientras que actualizar un registro requiere un bloqueo exclusivo. Los bloqueos se deshacen en los **puntos de sincronización**, es decir cuando se haga algún compromiso o algún retroceso.
Volvamos a analizar los problemas de concurrencias pero ahora con nuestra nueva herramienta.
El problema de la modificación perdida se puede solucionar simplemente al agregar estos

bloqueos. Cuando **A lee R**, se aplica un **bloqueo compartido** sobre este, tal que cuando B quiere leer puede también aplicar un bloqueo compartido sobre R.
Cuando **A actualiza R**, se intenta crear un **bloqueo exclusivo**, pero B tiene un bloqueo compartido sobre R por lo que A quedará en espera de que B suelte el bloqueo. Del mismo modo, cuando B intente actualizar R hará una solicitud de bloqueo exclusivo, pero A tiene un bloqueo compartido, por lo que B se quedará esperando a que A suelte el bloqueo. En este caso, solucionamos el problema de la modificación perdida, pero surge un nuevo problema. Si pensamos bien, A queda esperando a B y B queda esperando a A, es decir, ambas transacciones se bloquearon mutuamente. Este problema se llama, sorprendentemente, el problema del **bloqueo mutuo** (deadlock (enlace mortal)).

14.3. Bloqueo Mutuo

Podemos definir al **bloqueo mutuo** como la situación en la cual dos o mas transacciones entran en un estado de espera simultáneamente y cada uno espera que el otro libere el bloqueo para poder actuar. El sistema debe detectar y romper estas situaciones, eligiendo una transacción paralizada y cancelándola.

15. Bases de Datos Distribuidas

Una base de datos es distribuida cuando sus datos están dispersos en varias BD diferentes, administradas por varios motores y apoyadas por diversos sistemas operativos. Sin embargo, un usuario que quiera utilizar la base de datos será ignorante a esta fragmentación de los datos, y según su punto de vista, la base será una sola, como si un solo motor se encargara de ejecutar la base en una sola máquina. Decimos que esta forma de trabajar es ***transparente***.
Ahora, empecemos a dar mejores definiciones a estos conceptos por que así como esta no se entiende nada.
Cada una de estas "sub base de datos" se conocen como ***sitios***. Cada sitio es un sistema de base de datos en si mismo, cuenta con sus propias bases de datos, motores, usuarios, etc. De modo que una base de datos distribuida se reduce a básicamente muchos sitios trabajando en conjunto con el fin de que un usuario de cualquier sitio pueda acceder a los datos desde cualquier punto de la red como si los datos estuvieran almacenados en el sitio propio del usuario. Para lograr esta conexión y sincronización, aparece un nuevo componente que realiza las funciones de sociedad necesarias. La combinación entre este componente y los distintos motores de cada sitio se llama: "Sistema de Administración de bases de datos distribuidas (DDBMS)".

15.1. Ventajas y desventajas

Ahora veamos lo interesante:
Ventajas

- **Representación de la estructura de una empresa**: Normalmente una empresa que tenga una base de datos este subdividida tanto del punto de vista lógico (Divisiones, departamentos) como desde el punto de vista físico (Plantas, sucursales, talleres). De modo que la base de datos puede copiar esta estructura, manteniendo datos locales en forma local (en cada sitio) como también brindar la posibilidad de acceder a datos remotos (en otros sitios).
- **Se reduce al mínimo el número y volumen de los mensajes**: Los datos se encuentran normalmente en los sitios que los utilizan con mayor frecuencia, provocando que no necesiten comunicarse con ningún otro sitio.

Desventajas

- **Se requiere un buen diseño y organización**: En caso de no haberlo se convierte en un sistema demasiado complejo.
- Un sistema BDD no solo implica encargarse de los posibles problemas de las bases de datos locales de cada sitio, sino también posibles problemas en el sistema distribuido.
- **Puede aumentar costos iniciales**, tanto de Hardware como de Software.

15.2. Reglas

Para entender un sistema de bases de datos distribuidas se definen (*redoblantes*) ¡LAS 12 REGLAS! (en realidad son 13). Estas son:

- **REGLA 0 - Principio fundamental de las bases de datos distribuidas**
  Desde el punto de vista del usuario, un sistema distribuido deberá ser idéntico a un sistema no distribuido. Esto implica que las operaciones de **DML** no cambiaran, pero las operaciones de **DDL** deberán ser ampliadas para incluir operaciones como "crear la tabla en el sitio X y almacenarla en el sitio Y".
- **REGLA 1 - Autonomía local**
  Cada sitio debe ser **autónomo**, es decir ningún sitio X debe depender de otro sitio Y para su correcto funcionamiento.
- **REGLA 2 - Independencia de un sitio central**
  Todos los sitios deben tratarse por igual, no se debe depender de un sitio central para obtener un servicio.
- **REGLA 3 - Operación continua**
  Nunca debería haber necesidad de apagar el sistema para realizar alguna función (como para agregar un nuevo sitio).
- **REGLA 4 - Independencia con respecto a la localización**
  Los usuarios son ignorantes a la verdadera localización del almacenamiento de los datos.
- **REGLA 5 - Independencia con respecto a la fragmentación**
  Un sistema maneja una *fragmentación de los datos* si es posible dividir una relación en partes para propósitos de almacenamiento físico. Esto normalmente ayuda a reducir el trafico en la red, por ejemplo se almacenan los datos de los empleados de Rosario en el sitio de Rosario, ya que probablemente este sitio sea el que mas acceda a ellos.
  Se distinguen dos fragmentaciones:

1. **Horizontal**: Fragmentación **por filas**. Corresponde a una **selección**. Para reconstruir los datos fragmentados horizontalmente habrá que hacer una **unión**.
2. **Vertical**: Fragmentación **por columnas**. Corresponde a una **proyección**. Para reconstruir los datos fragmentados verticalmente habrá que hacer una **reunión**.

Tener una **independencia con respecto a la fragmentación** implica que los usuarios tengan una visión de los datos como si estuviesen combinados (mediantes reuniones y uniones).

- **REGLA 6 - Independencia de replica:**
  Una relación se puede representar en el nivel físico mediante varias copias almacenadas o en réplicas en muchos sitios distintos. De esta manera, las aplicaciones pueden funcionar sobre copias locales en vez de tener que comunicarse con sitios remotos. Sin embargo si un dato cambia, se deben actualizar todas sus réplicas.

- **REGLA 7 - Procesamiento distribuido de consultas**
  Existen optimizaciones a la hora de decidir cómo trasladar datos entre varios sitios. Según la estrategia el proceso puede ser mas o menos eficiente.

- **REGLA 8 - Manejo distribuido de transacciones**
  Cuando hablamos de manejo de transacciones hablamos de dos propiedades claves sobre estas:
  El **Control de recuperación** establece que una transacción debe ser **atómica**, es decir, que la transacción debe completarse en todos los agentes correspondientes a la transacción o fallar en todos.
  Y el **Control de concurrencia**, que, como lo indica el nombre, asegura la concurrencia de la transacción mediante sistemas como los locks (bloqueos).

- **REGLA 9 - Independencia con respecto al equipo**
  El sistema es independiente de los equipos en los cuales se esta ejecutando, ya que depende del motor.

- **REGLA 10 - Independencia con respecto al sistema operativo**
  Se debe poder ejecutar el mismo motor de sistemas distribuidos en diferentes equipos y sistemas operativos.

- **REGLA 11 - Independencia con respecto a la red**
  Se debe poder manejar varias redes de comunicación distintas entre los distintos sitios.

- **REGLA 12 - Independencia con respecto al motor**
  Los motores de sistemas distribuidos en los distintos sitios deben manejar la misma interfaz.

16. Bibliografía

- **Integridad**
  Slides de la cátedra: "Integridad"
- **Seguridad**
  Slides de la cátedra: "Seguridad"
- **Recuperación**
  Slides de la cátedra: "Recuperación"
  Database System Concepts - Silberschatz et. al - Sexta Edición
- **Concurrencia**
  Slides de la cátedra: "Concurrencia"
- **Sistemas de Bases de Datos distribuidas**
  Slides de la cátedra: "BDDistribuidas"